



Big Book of

Data

Engineering

3RD EDITION

Contents

- Introduction to Data Engineering on Databricks 3
- Guidance and Best Practices 13
 - Databricks Assistant Tips and Tricks for Data Engineers..... 14
 - Applying Software Development and DevOps Best Practices to Delta Live Tables Pipelines.....23
 - Unity Catalog Governance in Action: Monitoring, Reporting and Lineage.....33
 - Scalable Spark Structured Streaming for REST API Destinations..... 41
 - A Data Engineer’s Guide to Optimized Streaming With Protobuf and Delta Live Tables..... 48
 - Design Patterns for Batch Processing in Financial Services59
 - How to Set Up Your First Federated Lakehouse.....72
 - Orchestrating Data Analytics With Databricks Workflows78
 - Schema Management and Drift Scenarios via Databricks Auto Loader 84
 - From Idea to Code: Building With the Databricks SDK for Python.....97
- Ready-to-Use Notebooks and Datasets 105
- Case Studies107
 - Cox Automotive108
 - Block..... 111
 - Trek Bicycle..... 114
 - Coastal Community Bank..... 117
 - Powys Teaching Health Board (PTHB)..... 123

01

Introduction to Data Engineering on Databricks

Introduction to Data Engineering on Databricks

A recent **MIT Tech Review Report** shows that 88% of surveyed organizations are either investing in, adopting or experimenting with generative AI (GenAI) and 71% intend to build their own GenAI models. This increased interest in AI is fueling major investments as AI becomes a differentiating competitive advantage in every industry. As more organizations work to leverage their proprietary data for this purpose, many encounter the same hard truth:

The best GenAI models in the world will not succeed without good data.

This reality emphasizes the importance of building reliable data pipelines that can ingest or stream vast amounts of data efficiently and ensure high data quality. A unified platform and good data engineering are essential components of success in every data and AI initiative, especially in the era of GenAI.

Using practical guidance, useful patterns, best practices and real-world examples, this book will provide you with an understanding of how the **Databricks Data Intelligence Platform** helps data engineers meet the challenges of this new era.

What is data engineering?

Data engineering is the practice of taking raw data from a data source and processing it so it's stored and organized for a downstream use case such as data analytics, business intelligence (BI) or machine learning (ML) model training. In other words, it's the process of preparing data so value can be extracted from it.

A useful way of thinking about data engineering is by using the following framework, which includes three main parts:

1. Ingest

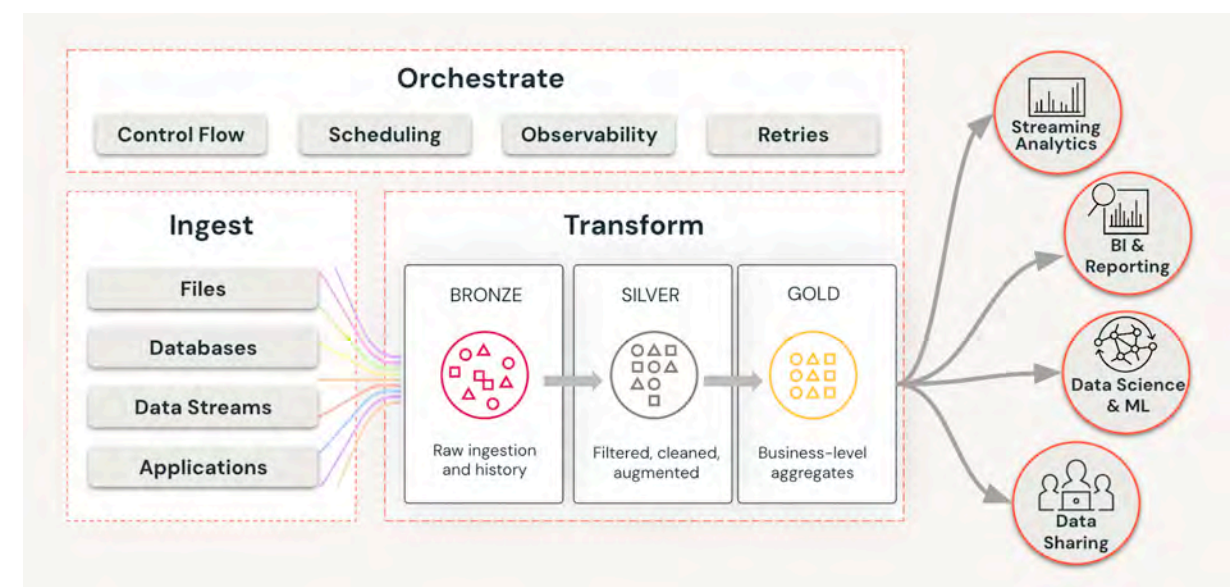
Data ingestion is the process of bringing data from one or more data sources into a data platform. These data sources can be files stored on-premises or on cloud storage services, databases, applications and, increasingly, data streams that produce real-time events.

2. Transform

Data transformation takes raw ingested data and uses a series of steps (referred to as “transformations”) to filter, standardize, clean and finally aggregate it so it's stored in a usable way. A popular pattern is the **medallion architecture**, which defines three stages in the process — Bronze, Silver and Gold.

3. Orchestrate

Data orchestration refers to the way a data pipeline that performs ingestion and transformation is scheduled and monitored as well as the control of the various pipeline steps and handling failures (e.g., by executing a retry run).



Challenges of data engineering in the AI era

As previously mentioned, data engineering is key to ensuring reliable data for AI initiatives. Data engineers who build and maintain ETL pipelines and the data infrastructure that underpins analytics and AI workloads face specific challenges in this fast-moving landscape.

- **Disparate data sources challenge most organizations:** ISG predicts that by 2026, 8 in 10 enterprises will have their data spread across multiple cloud providers and on-premises data centers that span multiple locations. This decentralization creates a dependency on specialized, siloed teams, inefficient pipelines and development with high costs, and slow time to value, which limits the usage of data and blocks innovation.
- **Handling real-time data:** From mobile applications to sensor data on factory floors, more and more data is created and streamed in real time and requires low-latency processing so it can be used in real-time decision-making.
- **Scaling data pipelines reliably:** With data coming in large quantities and often in real time, scaling the compute infrastructure that runs data pipelines is challenging, especially when trying to keep costs low and performance high. Running data pipelines reliably, monitoring data pipelines and troubleshooting when failures occur are some of the most important responsibilities of data engineers.
- **Data quality:** “Garbage in, garbage out.” High data quality is essential to training high-quality models and gaining actionable insights from data. Ensuring data quality is a key challenge for data engineers.

- **Governance and security:** Data governance is becoming a key challenge for organizations that find their data spread across multiple systems, with increasingly larger numbers of internal teams looking to access and utilize it for different purposes. Securing and governing data is also an important regulatory concern many organizations face, especially in highly regulated industries.

These challenges stress the importance of choosing the right data platform for navigating new waters in the age of AI. But a data platform in this new age can also go beyond addressing just the challenges of building AI solutions. The right platform can improve the experience and productivity of data practitioners, including data engineers, by infusing intelligence and using AI to assist with daily engineering tasks.

In other words, the new data platform is a data *intelligence* platform.

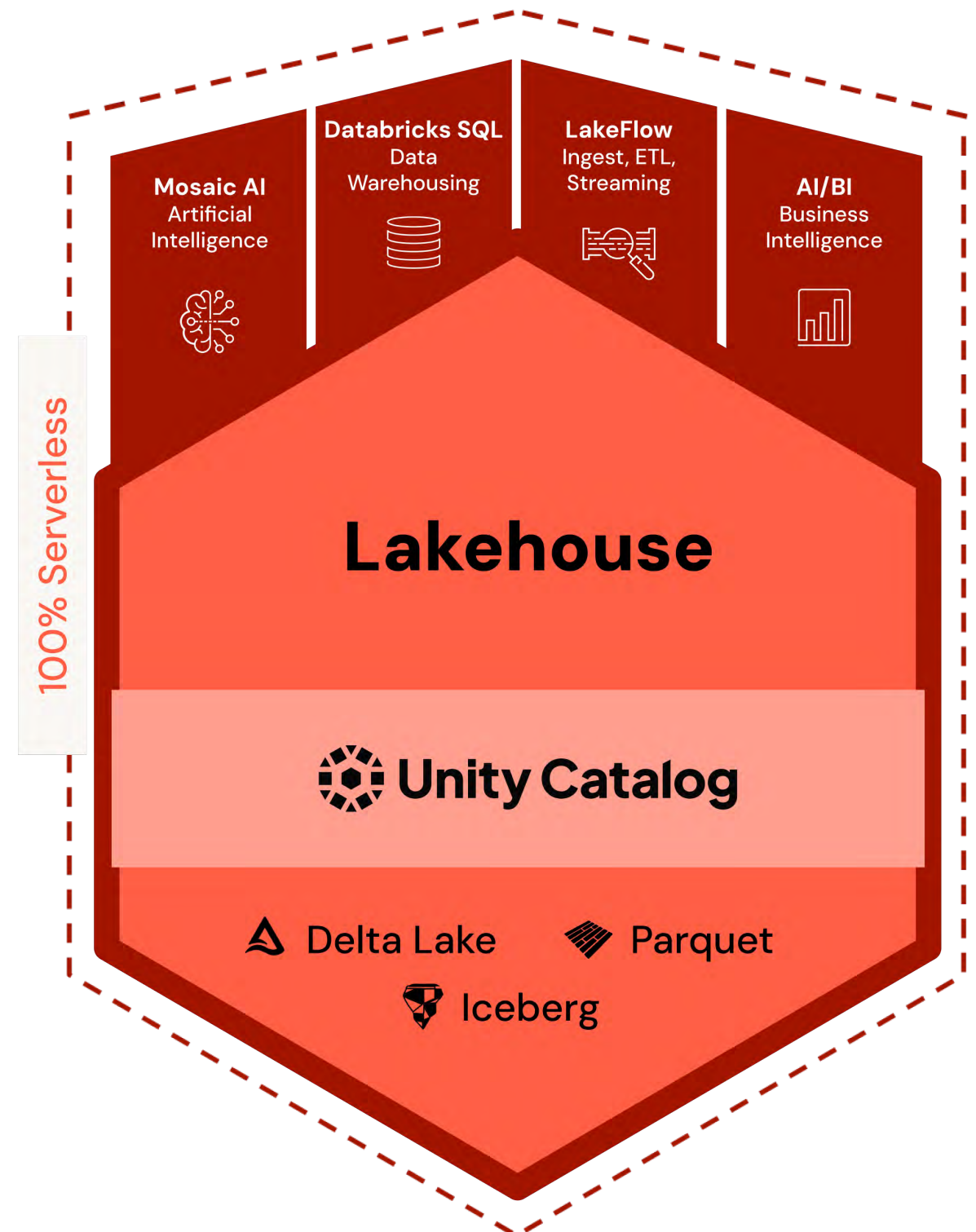
The Databricks Data Intelligence Platform

Databricks' mission is to democratize data and AI, allowing organizations to use their unique data to build or fine-tune their own machine learning and generative AI models so they can produce new insights that lead to business innovation.

The Databricks Data Intelligence Platform is built on a **lakehouse architecture** to provide an open, unified foundation for all data and governance, and is powered by a Data Intelligence Engine that understands the uniqueness of your data. With these capabilities at its foundation, the Data Intelligence Platform lets Databricks customers run a variety of workloads, from business intelligence and data warehousing to AI and data science.

To get a better understanding of the Databricks Platform, here's an overview of the different parts of the architecture as it relates to data engineering.

The Databricks Data Intelligence Platform enables you to execute all your data and AI initiatives. As a 100% serverless platform, it provides you with built-in features such as disaster recovery, cost controls and enterprise security. Key components feature Mosaic AI with end-to-end AI for both generative and classical AI; Databricks SQL, the most performant data warehouse in the cloud; efficient data ingestion and reliable transformation tools such as Workflows and Delta Live Tables (DLT) to ensure you can manage all your data for any workload; and AI/BI that integrates deeply with Databricks SQL to easily extend business intelligence across your business.



Data ingestion with Databricks LakeFlow

Databricks enables organizations to efficiently move data from various systems into a single, open and unified lakehouse architecture. Databricks **LakeFlow Connect** provides native data ingestion connectors for popular SaaS applications, databases and file sources that any practitioner can use to build incremental data pipelines at scale. These built-in connectors provide efficient end-to-end incremental ingestion, easy setup with a simple UI or API access, and governance via Unity Catalog — all powered by the Data Intelligence Platform. LakeFlow Connect is part of **LakeFlow** — Databricks' new unified data engineering solution spanning ingestion, transformation and orchestration — and is the first of these three components to roll out, compatible with existing tooling. In addition to LakeFlow Connect, Databricks Auto Loader, a connector for cloud object storage, is compatible with Structured Streaming and Delta Live Tables.

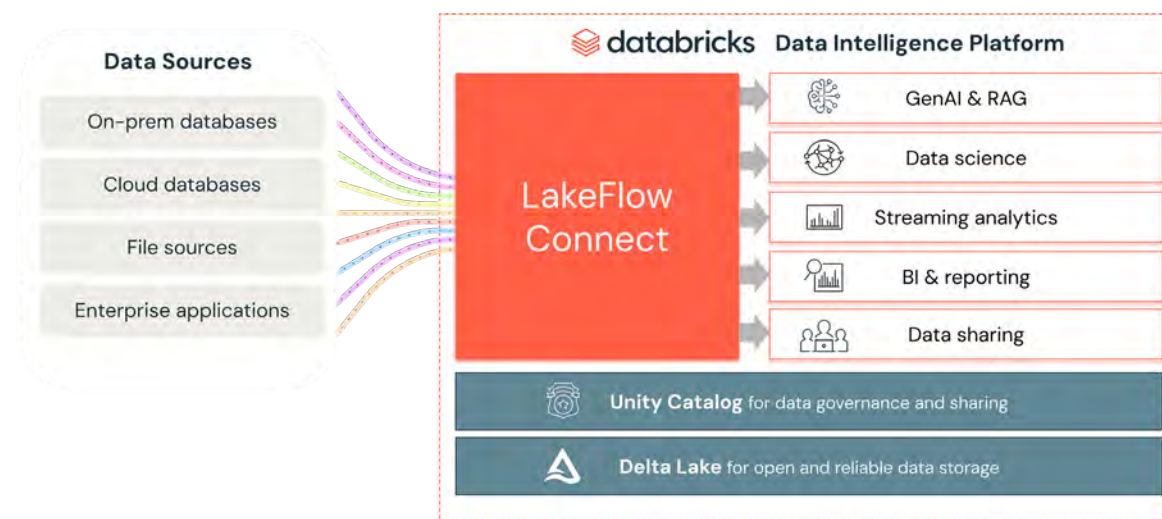
Data reliability and performance with Delta Lake

To bring openness, reliability and lifecycle management to data lakes, the Databricks lakehouse architecture is built on the foundation of **Delta Lake**, an open source, highly performant storage format that solves challenges around unstructured/structured data ingestion, the application of data quality, difficulties with deleting data for compliance or issues with modifying data for data capture. Delta Lake UniForm users can now read Delta tables with Hudi and Apache Iceberg™ clients, keeping them in control of their data. In addition, **Delta Sharing** enables easy and secure sharing of datasets inside and outside the organization.

Unified governance with Unity Catalog

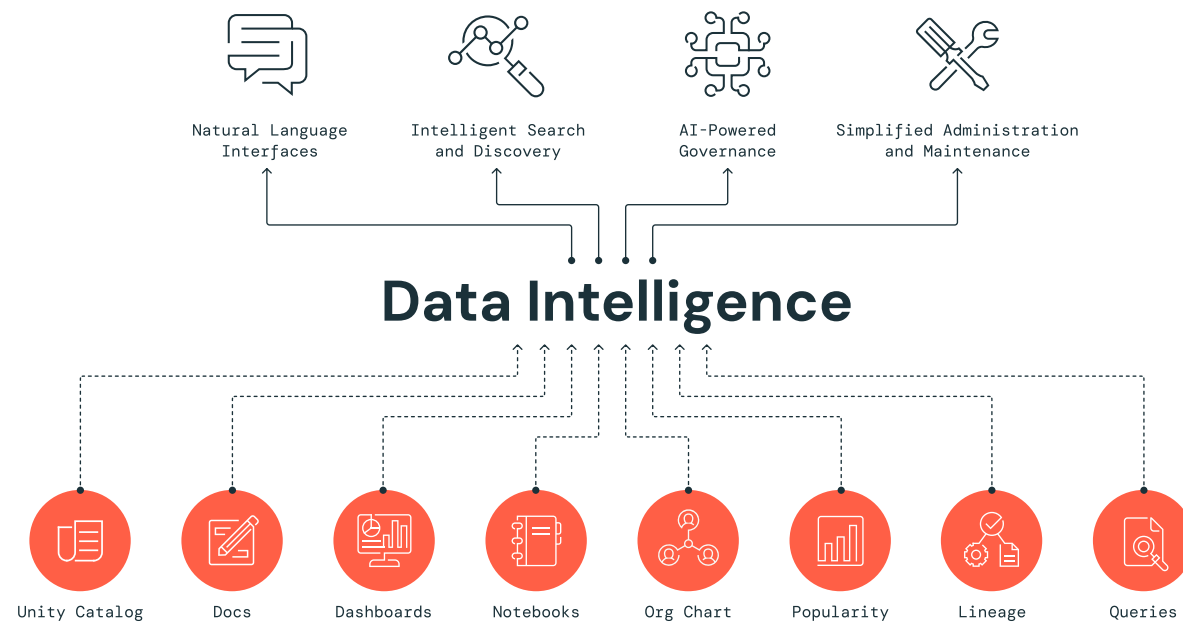
With **Unity Catalog**, data engineering and governance teams benefit from an enterprisewide data catalog with a single interface to manage permissions, centralize auditing, automatically track data lineage down to the column level and share data across platforms, clouds and regions.

Native ingestion for the Data Intelligence Platform



Accelerate productivity through Data Intelligence

Databricks brings AI to your data to help you bring AI to the world, and at the heart of the Databricks Platform is **Data Intelligence**. Databricks helps you succeed with AI with your own data to democratize insights and drive down costs. Databricks leverages generative AI with Data Intelligence to power all parts of the platform. Using signals across your entire Databricks environment, including Unity Catalog, dashboards, notebooks, data pipelines and documentation, the Data Intelligence Engine creates highly specialized and accurate generative AI models that understand your data, your usage patterns and your business terminology.



Reliable data pipelines and real-time stream processing with Delta Live Tables

Delta Live Tables (DLT) is a declarative ETL framework that helps data teams simplify and make ETL cost-effective in streaming and batch. Simply define the transformations you want to perform on your data and let DLT pipelines automatically handle task orchestration, cluster management, monitoring, data quality and error management. Engineers can treat their data as code and apply modern software engineering best practices like testing, error handling, monitoring and documentation to deploy reliable pipelines at scale. DLT fully supports both Python and SQL and is tailored to work with both streaming and batch workloads.

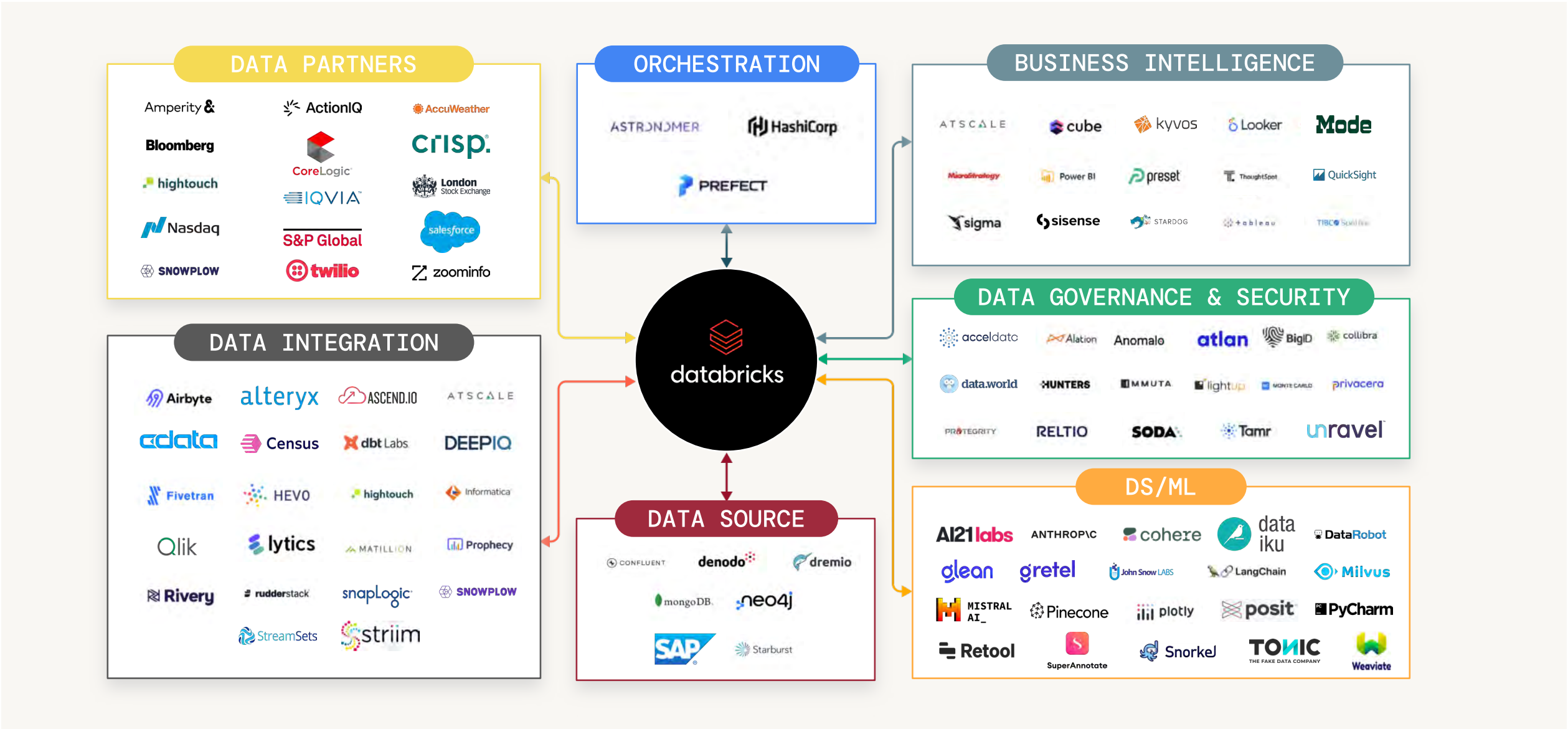
Unified data orchestration with Databricks Workflows

Databricks Workflows offers a simple, reliable orchestration solution for data and AI on the Data Intelligence Platform. Databricks Workflows lets you define multistep workflows to implement ETL pipelines, ML training workflows and more. It offers enhanced control flow capabilities and supports different task types and workflow triggering options. As the platform native orchestrator, Databricks Workflows also provides advanced observability to monitor and visualize workflow execution along with alerting capabilities for when issues arise. Databricks Workflows offers serverless compute options so you can leverage smart scaling and efficient task execution.

A rich ecosystem of data solutions

The Data Intelligence Platform is built on open source technologies and uses open standards so leading data solutions can be leveraged with anything you build on the lakehouse.

A large collection of **technology partners** makes it easy and simple to integrate the technologies you rely on when migrating to Databricks — and you are not locked into a closed data technology stack.

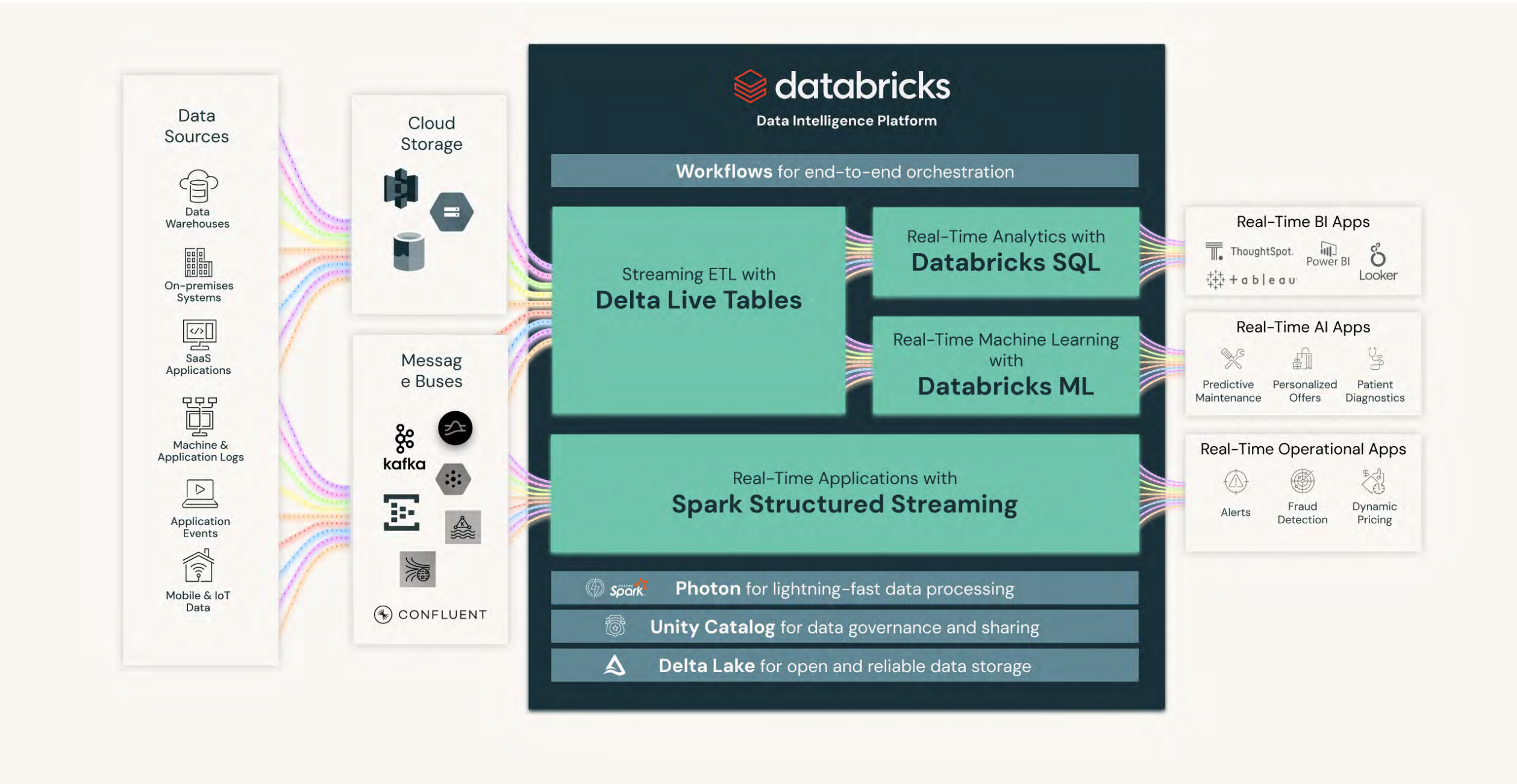


The Data Intelligence Platform integrates with a large collection of technologies

Why data engineers choose the Data Intelligence Platform

So how does the Data Intelligence Platform help with each of the data engineering challenges discussed earlier?

- **Efficient ingestion, wide range of data connectors:** Databricks allows you to efficiently ingest data, only bringing in new data or table updates. With a growing set of native connectors for popular data sources, as well as a broad network of data ingestion partners, you can easily move data from siloed systems into your data platform. Ingesting and storing your data in Delta Lake while leveraging the reliability and scalability of the Data Intelligence Platform is the first step in extracting value from your data and accelerating innovation.
- **Real-time data stream processing:** The Data Intelligence Platform simplifies development and operations by automating the production aspects associated with building and maintaining real-time data workloads. Delta Live Tables provides a declarative way to define streaming ETL pipelines, and Spark Structured Streaming helps build real-time applications for real-time decision-making (see diagram on next page).

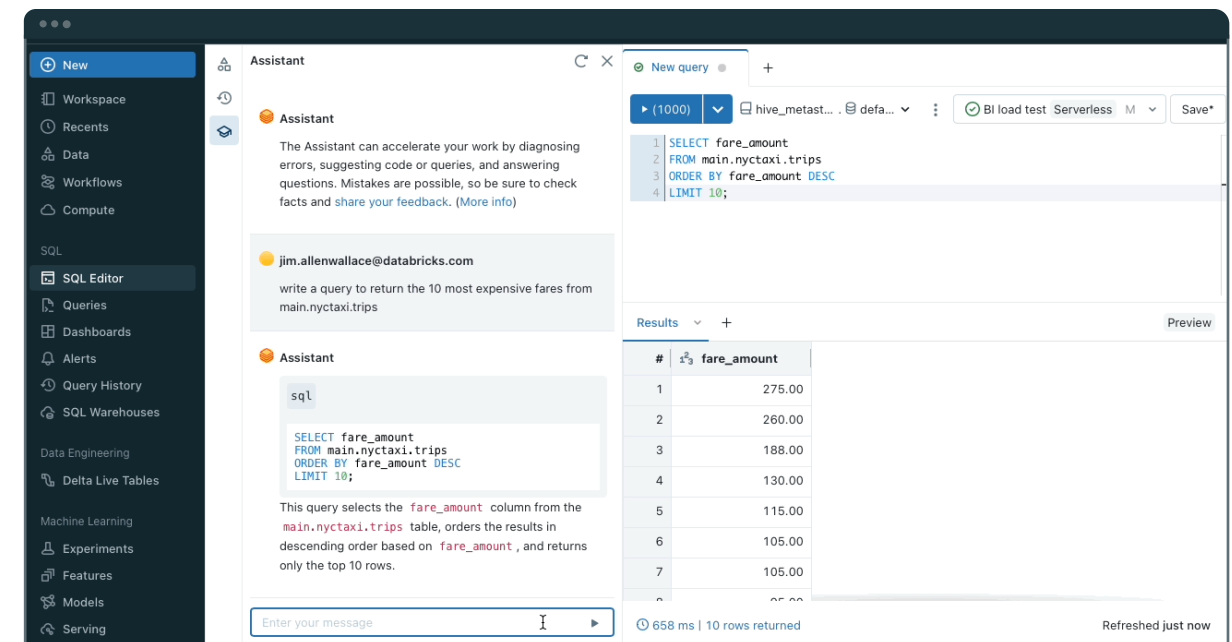


A unified set of tools for real-time data processing

- **Reliable data pipelines at scale:** Both **Delta Live Tables** and **Databricks Workflows** use smart autoscaling and auto-optimized resource management to handle high-scaled workloads. With lakehouse architecture, the high scalability of data lakes is combined with the high reliability of data warehouses, thanks to Delta Lake — the storage format that sits at the foundation of the lakehouse.
- **Data quality:** High reliability — starting at the storage level with **Delta Lake** and coupled with data quality-specific features offered by Delta Live Tables — ensures high data quality. These features include setting data “expectations” to handle corrupt or missing data as well as automatic retries. In addition, both Databricks Workflows and Delta Live Tables provide full observability to data engineers, making issue resolution faster and easier.
- **Unified governance with secure data sharing:** **Unity Catalog** provides a single governance model for the entire platform so every dataset and pipeline are governed in a consistent way. Datasets are discoverable and can be securely shared with internal or external teams using Delta Sharing. In addition, because Unity Catalog is a cross-platform governance solution, it provides valuable lineage information, so it’s easy to have a full understanding of how each dataset and table is used downstream and where it originates upstream.

In addition, data engineers using the Data Intelligence Platform benefit from cutting-edge innovations in the form of GenAI-infused intelligence:

- **AI-powered productivity:** Specifically useful for data engineers, the **Databricks Assistant** is a context-aware AI assistant that offers a conversational API to query data, generate code, explain code queries and even fix issues.



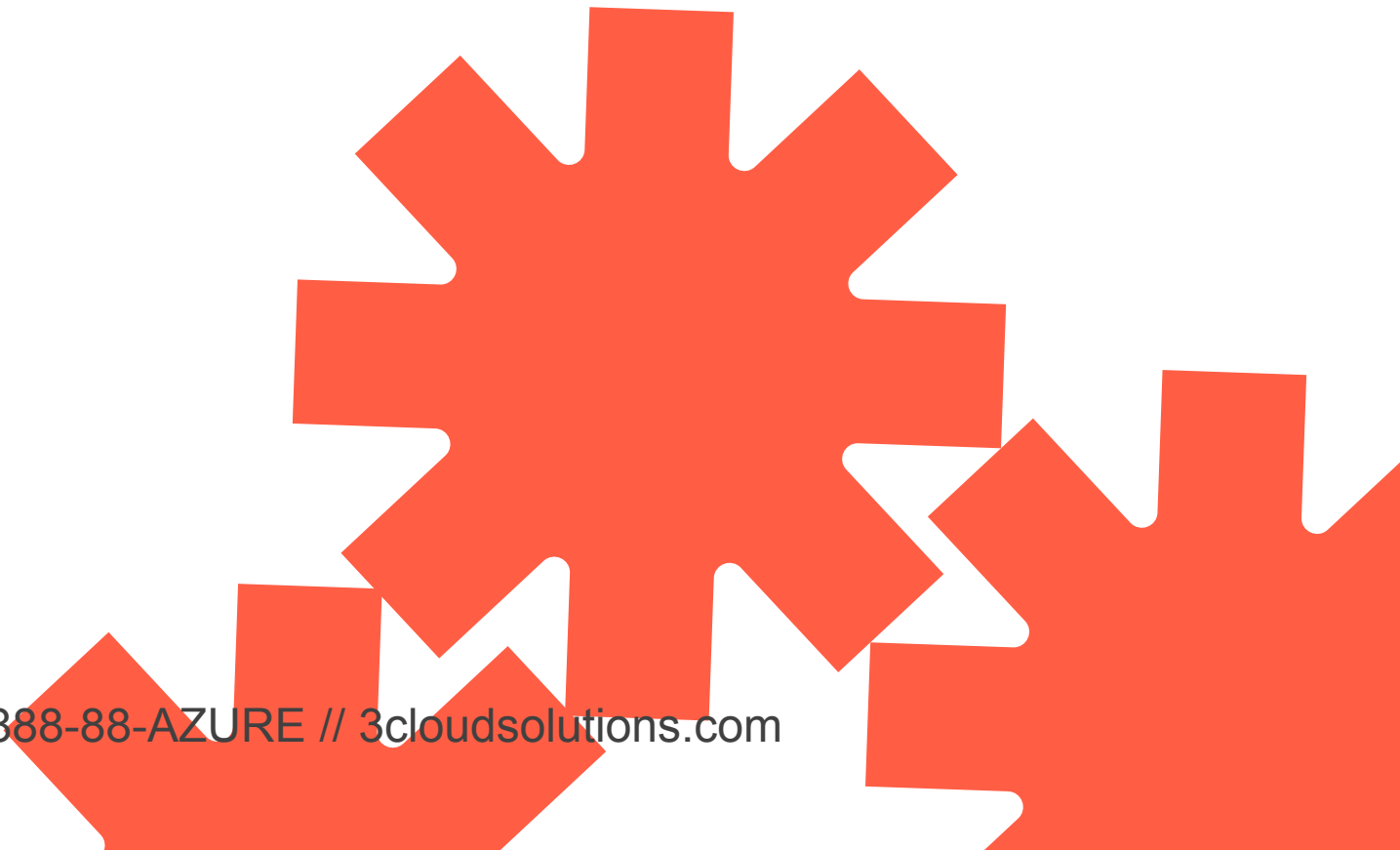
Conclusion

As organizations strive to innovate with AI, data engineering is a focal point for success by delivering reliable, real-time data pipelines that make AI possible. With the Databricks Platform, built on a lakehouse architecture and powered by Data Intelligence, data engineers are set up for success in dealing with the critical challenges posed in the modern data landscape. By leaning on the advanced capabilities of the Data Intelligence Platform, data engineers don't need to spend as much time managing complex pipelines or dealing with reliability, scalability and data quality issues. Instead, they can focus on innovation and bringing more value to the organization.

FOLLOW PROVEN BEST PRACTICES

In the next section, we describe best practices for data engineering and end-to-end use cases drawn from real-world examples. From data ingestion and real-time processing to orchestration and data federation, you'll learn how to apply proven patterns and make the best use of the different capabilities of the Data Intelligence Platform.

As you explore the rest of this guide, you can find datasets and code samples in the various [Databricks Solution Accelerators](#), so you can get your hands dirty and start building on the Data Intelligence Platform.



02

Guidance and Best Practices

Databricks Assistant Tips and Tricks for Data Engineers

by Jackie Zhang, Rafi Kurlansik and Richard Tomlinson

The generative AI revolution is transforming the way that teams work, and Databricks Assistant leverages the best of these advancements. It allows you to query data through a conversational interface, making you more productive inside your Databricks Workspace. The Assistant is powered by DatabricksIQ, the Data Intelligence Engine for Databricks, helping to ensure your data is secured and responses are accurate and tailored to the specifics of your enterprise. Databricks Assistant lets you describe your task in natural language to generate, optimize, or debug complex code without interrupting your developer experience.

In this chapter we'll discuss how to get the most out of your Databricks Assistant and focus on how the Assistant can improve the life of Data Engineers by eliminating tedium, increasing productivity and immersion, and accelerating time to value. We will follow up with a series of posts focused on different data practitioner personas, so stay tuned for upcoming entries focused on data scientists, SQL analysts, and more.

INGESTION

When working with Databricks as a data engineer, ingesting data into Delta Lake tables is often the first step. Let's take a look at two examples of how the Assistant helps load data, one from APIs, and one from files in cloud storage. For each, we will share the prompt and results. As mentioned in the [5 tips blog](#), being specific in a prompt gives the best results, a technique consistently used in this article.

To get data from the datausa.io API and load it into a Delta Lake table with Python, we used the following prompt:

Help me ingest data from this API into a Delta Lake table:

<https://datausa.io/api/data?drilldowns=Nation&measures=Population>

Make sure to use PySpark, and be concise! If the Spark DataFrame columns have any spaces in them, make sure to remove them from the Spark DF.

```

import requests

# Fetch the data from the API
api_url = "https://datausa.io/api/data?drilldowns=Nation&measures=Population"
response = requests.get(api_url)
data = response.json()

# Create a Spark DataFrame from the API response data
df = spark.createDataFrame(data["data"])

# Remove spaces from column names in the DataFrame
df = df.toDF(*[col.replace(" ", "") for col in df.columns])

# Specify the Delta Lake table path
table_path = "/Volumes/rkurlansik/default/data_science/us_data"

# Write the DataFrame as a Delta Lake table
df.write.format("delta").mode("overwrite").save(table_path)

display(df)

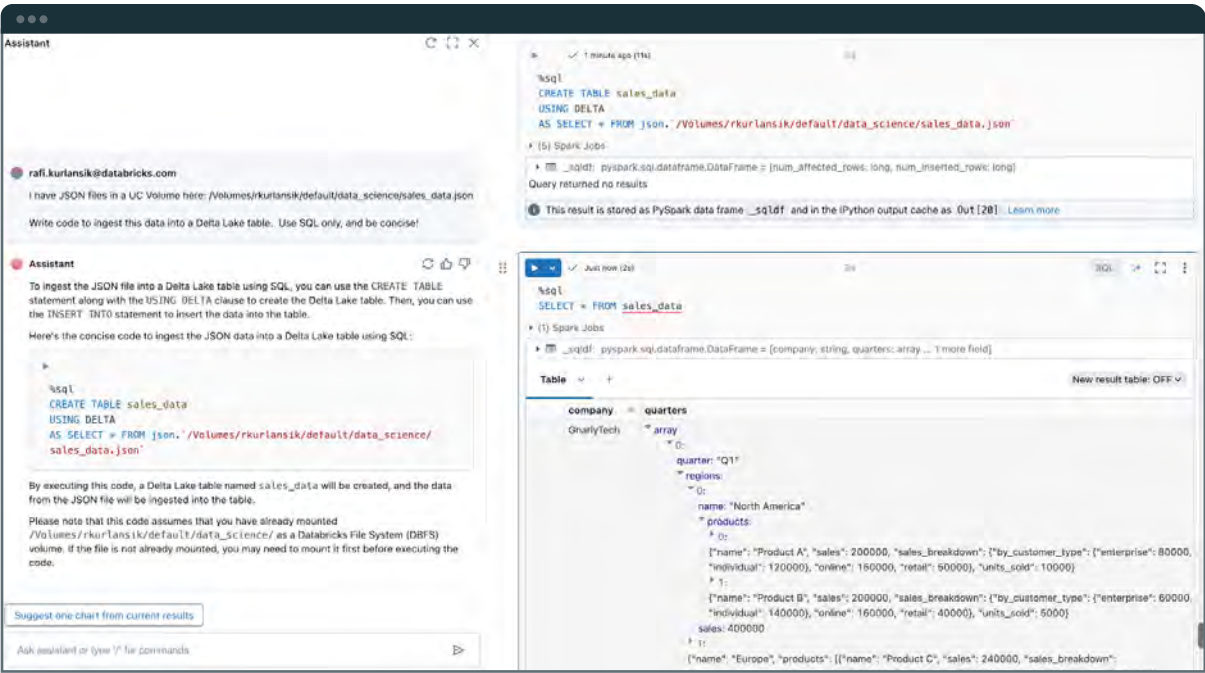
```

IDNation	IDYear	Nation	Population	SlugNation	Year
01000US	2021	United States	329725481	united-states	2021
01000US	2020	United States	326569308	united-states	2020
01000US	2019	United States	324697795	united-states	2019
01000US	2018	United States	322903030	united-states	2018
01000US	2017	United States	321004407	united-states	2017
01000US	2016	United States	318558162	united-states	2016

A similar prompt can be used to ingest JSON files from cloud storage into Delta Lake tables, this time using SQL:

I have JSON files in a UC Volume here: /Volumes/rkurlansik/default/data_science/sales_data.json

Write code to ingest this data into a Delta Lake table. Use SQL only, and be concise!



TRANSFORMING DATA FROM UNSTRUCTURED TO STRUCTURED

Following tidy data principles, any given cell of a table should contain a single observation with a proper data type. Complex strings or nested data structures are often at odds with this principle, and as a result, data engineering work consists of extracting structured data from unstructured data. Let’s explore two examples where the Assistant excels at this task — using regular expressions and exploding nested data structures.

Regular expressions

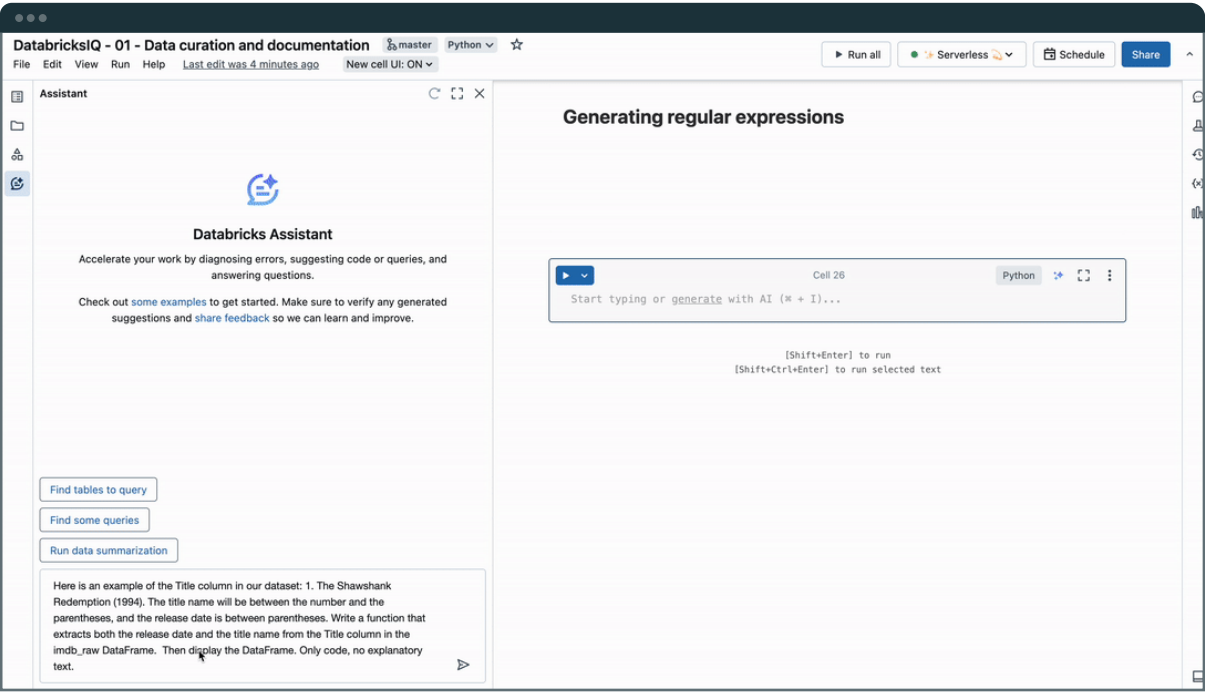
Regular expressions are a means to extract structured data from messy strings, but figuring out the correct regex takes time and is tedious. In this respect, the Assistant is a boon for all data engineers who struggle with regex.

Consider this example using the *Title* column from the IMDb dataset:

Title	Certificate	Duration	Genre
1. The Shawshank Redemption (1994)	R	142 min	Drama

This column contains two distinct observations — film title and release year. With the following prompt, the Assistant identifies an appropriate regular expression to parse the string into multiple columns.

Here is an example of the Title column in our dataset: 1. The Shawshank Redemption (1994). The title name will be between the number and the parentheses, and the release date is between parentheses. Write a function that extracts both the release date and the title name from the Title column in the imdb_raw DataFrame.



Providing an example of the string in our prompt helps the Assistant find the correct result. If you are working with sensitive data, we recommend creating a fake example that follows the same pattern. In any case, now you have one less problem to worry about in your data engineering work.

Nested Structs, Arrays (JSON, XML, etc)

When ingesting data via API, JSON files in storage, or noSQL databases, the resulting Spark DataFrames can be deeply nested and tricky to flatten correctly. Take a look at this mock sales data in JSON format:

```
df = spark.read.json("/Volumes/rkurlansik/default/data_science/sales_data.json")
display(df)
```

(2) Spark Jobs

df: pyspark.sql.dataframe.DataFrame = [company: string, quarters: array ... 1 more field]

Table

company	quarters	year
GnarlyTech	array	2024

1 row | 51.40 seconds runtime | Refreshed 1 minute ago

Data engineers may be asked to flatten the nested array and extract revenue metrics for each product. Normally this task would take significant trial and error — even in a case where the data is relatively straightforward. The Assistant, however, being context-aware of the schemas of DataFrames you have in memory, generates code to get the job done. Using a simple prompt, we get the results we are looking for in seconds.

Write PySpark code to flatten the df and extract revenue for each product and customer

Nested JSON

File Edit View Run Help Last edit was 1 minute ago New cell UI: ON

Assistant

Accelerate your work by diagnosing errors, suggesting code or queries, and answering questions.

Check out some examples to get started. Make sure to verify any generated suggestions and share feedback so we can learn and improve.

Find tables to query

Find some queries

Write PySpark code to flatten the df and extract revenue for each product and customer

```
df = spark.read.json("/Volumes/rkurlansik/default/data_science/sales_data.json")
display(df)
```

(2) Spark Jobs

df: pyspark.sql.dataframe.DataFrame = [company: string, quarters: array ... 1 more field]

Table

company	quarters	year
GnarlyTech	array	2024

1 row | 1.22 seconds runtime | Refreshed now

REFACTORING, DEBUGGING AND OPTIMIZATION

Another scenario data engineers face is rewriting code authored by other team members, either ones that may be more junior or have left the company. In these cases, the Assistant can analyze and explain poorly written code by understanding its context and intent. It can suggest more efficient algorithms, refactor code for better readability, and add comments.

Improving documentation and maintainability

This Python code calculates the total cost of items in an online shopping cart.

```

1  def calculate_total(cart_items):
2      total = 0
3      for i in range(len(cart_items)):
4          if cart_items[i]['type'] == 'book':
5              discount = cart_items[i]['price'] * 0.05
6              total += cart_items[i]['price'] - discount
7          else:
8              total += cart_items[i]['price']
9      return total

10  cart_items = [{'name': 'Python Programming', 'type': 'book', 'price': 50},
11                {'name': 'Laptop', 'type': 'electronics', 'price': 800}]
12  total_price = calculate_total(cart_items)

```

The use of conditional blocks in this code makes it hard to read and inefficient at scale. Furthermore, there are no comments to explain what is happening. A good place to begin is to ask the Assistant to explain the code step by step. Once the data engineer understands the code, the Assistant can transform it, making it more performant and readable with the following prompt:

Rewrite this code in a way that is more performant, commented properly, and documented according to Python function documentation standards

The generated example below properly documents the code, and uses generator expressions instead of conditional blocks to improve memory utilization on larger datasets.

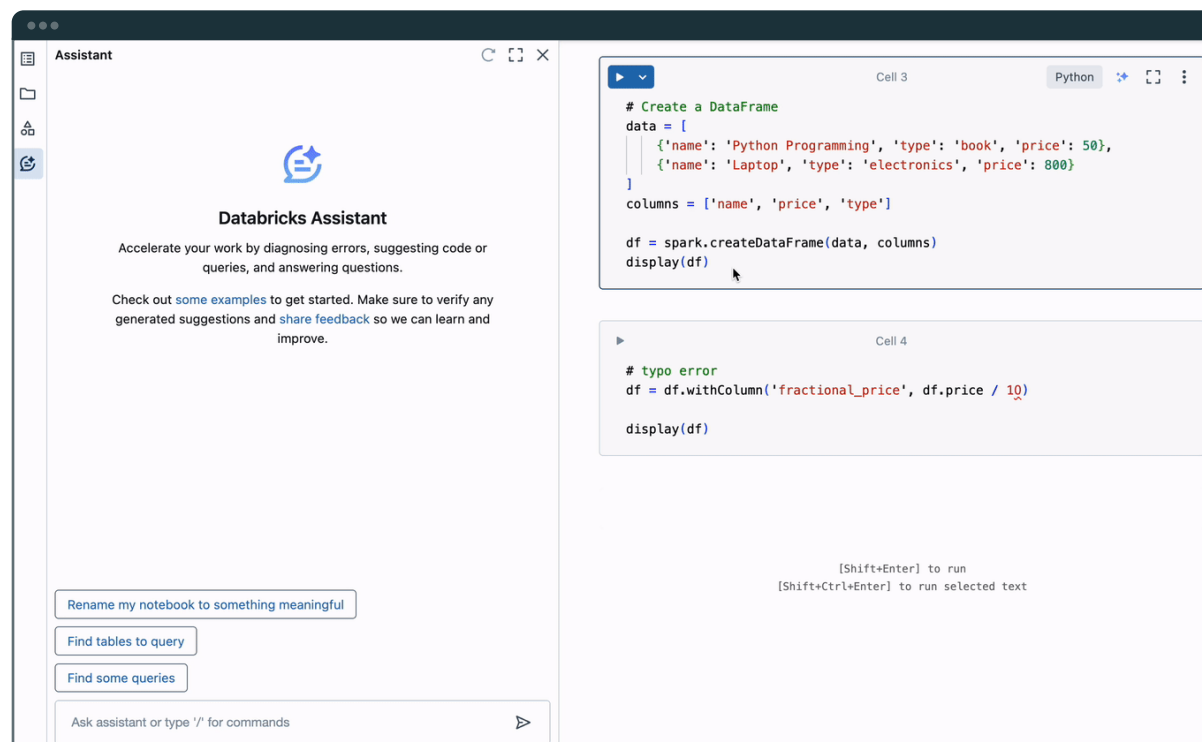
```

1  def calculate_total(cart_items):
2      """
3      Calculate the total price of items in the shopping cart.
4
5      Apply a 5% discount to book items. The total is calculated by summing
6      the prices, taking into account the discount for books.
7
8      Parameters:
9      cart_items (list of dict): A list of dictionaries, where each dictionary
10                                 represents an item in the cart with keys 'name',
11                                 'type', and 'price'.
12
13      Returns:
14      float: The total price of all items in the cart after discounts.
15      """
16      # Use a generator expression to iterate over cart items, applying a 5% discount
17      # to books.
18      return sum(item['price'] * 0.95 if item['type'] == 'book' else item['price'] for
19                item in cart_items)
20
21  # Example usage
22  cart_items = [{'name': 'Python Programming', 'type': 'book', 'price': 50},
23                {'name': 'Laptop', 'type': 'electronics', 'price': 800}]
24  total_price = calculate_total(cart_items)

```

Diagnosing errors

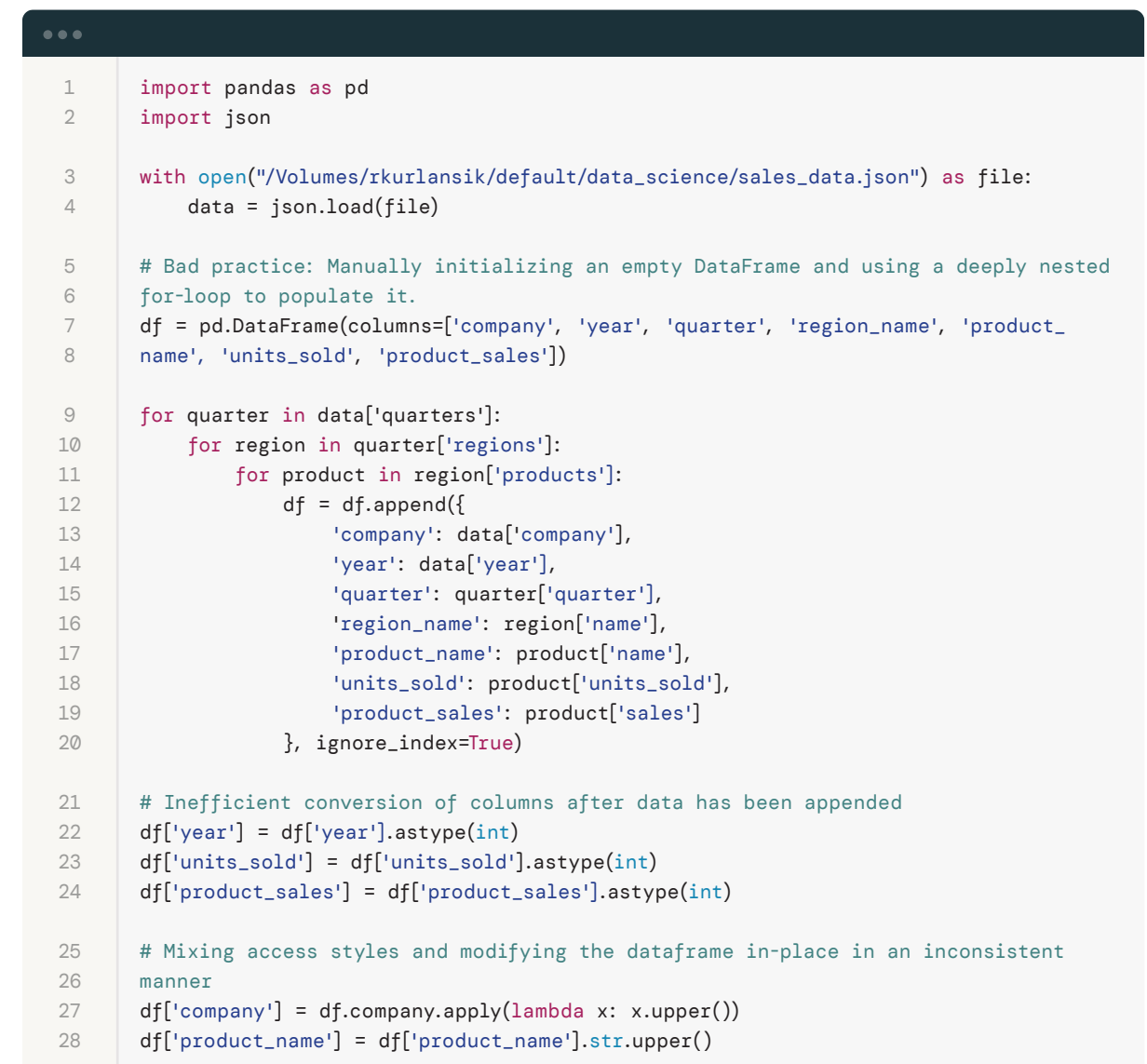
Inevitably, data engineers will need to debug. The Assistant eliminates the need to open multiple browser tabs or switch contexts in order to identify the cause of errors in code, and staying focused is a tremendous productivity boost. To understand how this works with the Assistant, let's create a simple PySpark DataFrame and trigger an error.



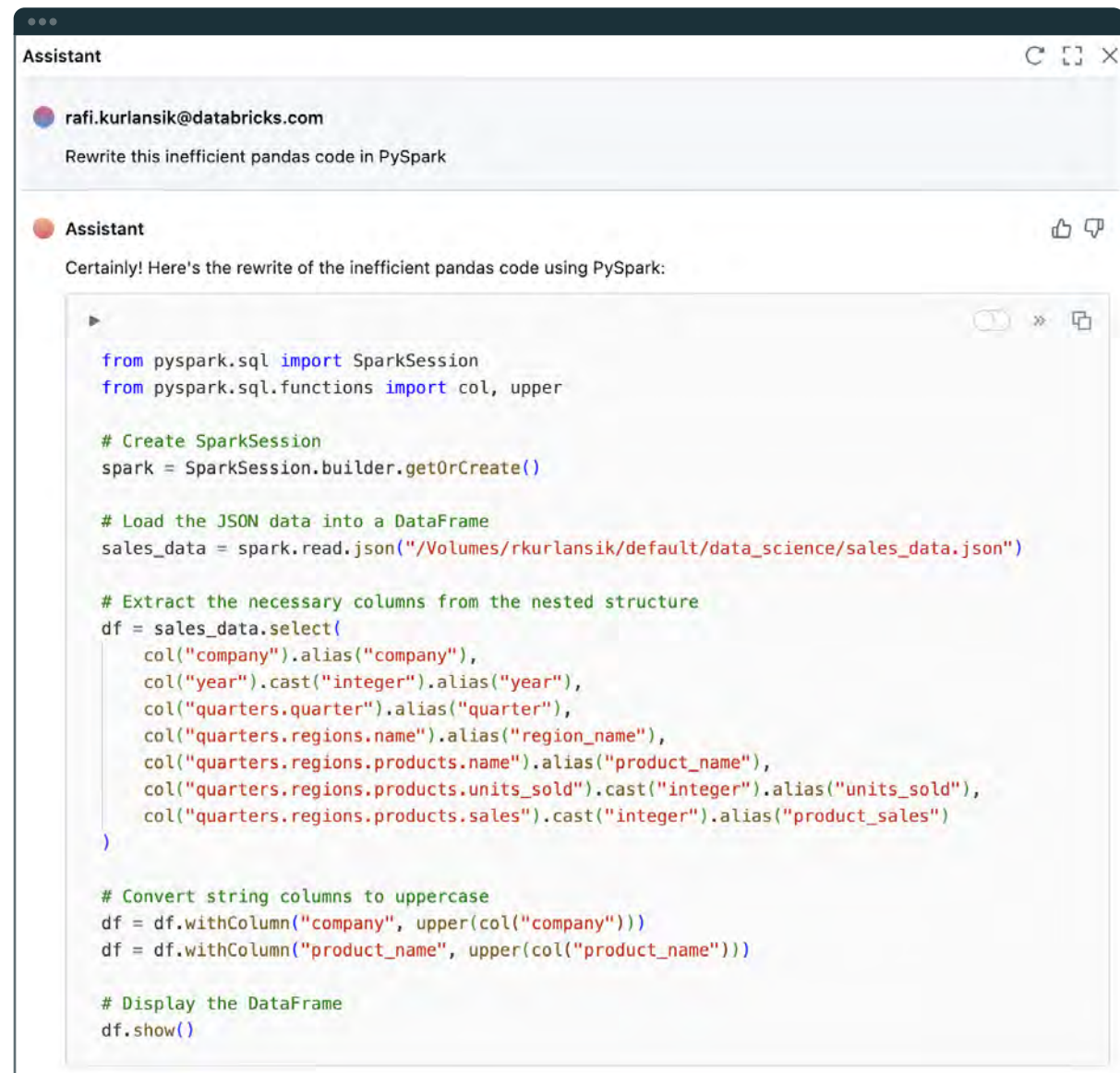
In the above example, a typo is introduced when adding a new column to the DataFrame. The zero in "10" is actually the letter "O", leading to an *invalid decimal literal* syntax error. The Assistant immediately offers to diagnose the error. It correctly identifies the typo, and suggests corrected code that can be inserted into the editor in the current cell. Diagnosing and correcting errors this way can save hours of time spent debugging.

Transpiling pandas to PySpark

Pandas is one of the most successful data-wrangling libraries in Python and is used by data scientists everywhere. Sticking with our JSON sales data, let's imagine a situation where a novice data scientist has done their best to flatten the data using pandas. It isn't pretty, it doesn't follow best practices, but it produces the correct output:



By default, Pandas is limited to running on a single machine. The data engineer shouldn't put this code into production and run it on billions of rows of data until it is converted to PySpark. This conversion process includes ensuring the data engineer understands the code and rewrites it in a way that is maintainable, testable, and performant. The Assistant once again comes up with a better solution in seconds.



Assistant

rafi.kurlansik@databricks.com

Rewrite this inefficient pandas code in PySpark

Assistant

Certainly! Here's the rewrite of the inefficient pandas code using PySpark:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, upper

# Create SparkSession
spark = SparkSession.builder.getOrCreate()

# Load the JSON data into a DataFrame
sales_data = spark.read.json("/Volumes/rkurlansik/default/data_science/sales_data.json")

# Extract the necessary columns from the nested structure
df = sales_data.select(
    col("company").alias("company"),
    col("year").cast("integer").alias("year"),
    col("quarters.quarter").alias("quarter"),
    col("quarters.regions.name").alias("region_name"),
    col("quarters.regions.products.name").alias("product_name"),
    col("quarters.regions.products.units_sold").cast("integer").alias("units_sold"),
    col("quarters.regions.products.sales").cast("integer").alias("product_sales")
)

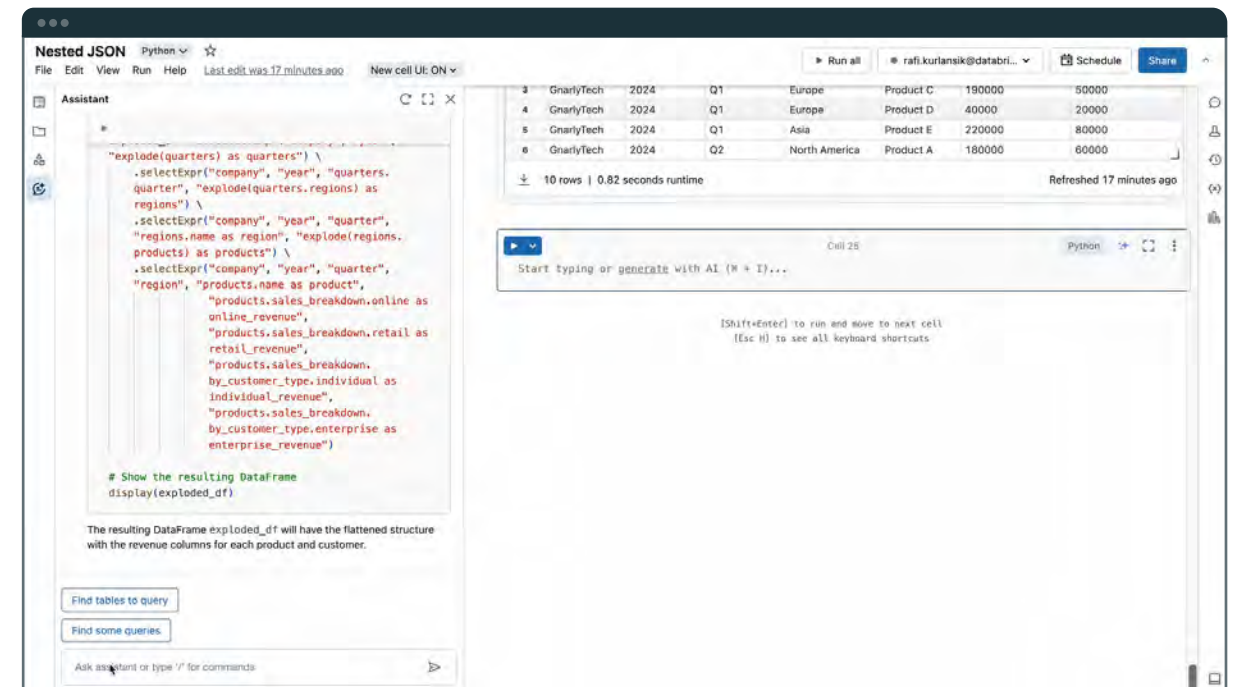
# Convert string columns to uppercase
df = df.withColumn("company", upper(col("company")))
df = df.withColumn("product_name", upper(col("product_name")))

# Display the DataFrame
df.show()
```

Note the generated code includes creating a *SparkSession*, which isn't required in Databricks. Sometimes the Assistant, like any LLM, can be wrong or hallucinate. You, the data engineer, are the ultimate author of your code and it is important to review and understand any code generated before proceeding to the next task. If you notice this type of behavior, adjust your prompt accordingly.

WRITING TESTS

One of the most important steps in data engineering is to write tests to ensure your DataFrame transformation logic is correct, and to potentially catch any corrupted data flowing through your pipeline. Continuing with our example from the JSON sales data, the Assistant makes it a breeze to test if any of the revenue columns are negative – as long as values in the revenue columns are not less than zero, we can be confident that our data and transformations in this case are correct.



Nested JSON Python

File Edit View Run Help Last edit was 17 minutes ago New cell UI: ON

Assistant

```
"explode(quarters) as quarters" \
.selectExpr("company", "year", "quarters.
quarter", "explode(quarters.regions) as
regions") \
.selectExpr("company", "year", "quarter",
"regions.name as region", "explode(regions.
products) as products") \
.selectExpr("company", "year", "quarter",
"region", "products.name as product",
"products.sales_breakdown.online as
online_revenue",
"products.sales_breakdown.retail as
retail_revenue",
"products.sales_breakdown.
by_customer_type.individual as
individual_revenue",
"products.sales_breakdown.
by_customer_type.enterprise as
enterprise_revenue")

# Show the resulting DataFrame
display(exploded_df)
```

The resulting DataFrame exploded_df will have the flattened structure with the revenue columns for each product and customer.

Find tables to query

Find some queries

Ask assistant or type '/' for commands

	Company	Year	Quarter	Region	Product	Units Sold	Product Sales
3	GnarlyTech	2024	Q1	Europe	Product C	190000	50000
4	GnarlyTech	2024	Q1	Europe	Product D	40000	20000
5	GnarlyTech	2024	Q1	Asia	Product E	220000	80000
6	GnarlyTech	2024	Q2	North America	Product A	180000	60000

10 rows | 0.82 seconds runtime

Refreshed 17 minutes ago

Cell 28

Python

Start typing or generate with AI (M + I)...

[Shift+Enter] to run and move to next cell
[Esc H] to see all keyboard shortcuts

We can build off this logic by asking the Assistant to incorporate the test into PySpark's native testing functionality, using the following prompt:

Write a test using `assertDataFrameEqual` from `pyspark.testing.utils` to check that an empty DataFrame has the same number of rows as our negative revenue DataFrame.

The Assistant obliges, providing working code to bootstrap our testing efforts.

The screenshot shows the Databricks Assistant interface. On the left, the Assistant's response to the prompt is displayed, including instructions and a code snippet. The code snippet is as follows:

```
from pyspark.sql import SparkSession
from pyspark.testing.utils import assertDataFrameEqual

spark = SparkSession.builder.getOrCreate()

# Create empty DataFrame with the same schema as the negative revenue DataFrame
empty_df = spark.createDataFrame([], schema=negative_revenue_df.schema)

# Assert that the empty DataFrame and the negative revenue DataFrame have the same number of rows
try:
    assertDataFrameEqual(empty_df, negative_revenue_df)
except AssertionError:
    raise AssertionError("The DataFrames do not have the same number of rows")
```

Below the code, there is a note: "Note that you need to replace `negative_revenue_df` with the actual". At the bottom, there is a prompt: "Ask assistant or type '/' for commands".

This example highlights the fact that being specific and adding detail to your prompt yields better results. If we simply ask the Assistant to write tests for us without any detail, our results will exhibit more variability in quality. Being specific and clear in what we are looking for — a test using PySpark modules that builds off the logic it wrote — generally will perform better than assuming the Assistant can correctly guess at our intentions.

GETTING HELP

Beyond a general capability to improve and understand code, the Assistant possesses knowledge of the entire Databricks documentation and Knowledge Base. This information is indexed on a regular basis and made available as additional context for the Assistant via a RAG architecture. This allows users to search for product functionality and configurations without leaving the Databricks Platform.

For example, if you want to know details about the system environment for the version of Databricks Runtime you are using, the Assistant can direct you to the appropriate page in the Databricks documentation.

The screenshot shows the Databricks Assistant interface. At the top, the user's email address is displayed: **rafi.kurlansik@databricks.com**. Below this, the user's query is shown: "I need to know more about the versions of libraries in the most recent Databricks Runtime, can you give me a link?". The Assistant's response is displayed below the query, starting with the Assistant's name and a detailed answer. The answer includes a link to the Databricks Runtime Release Notes: <https://docs.databricks.com/release-notes/index.html>. The Assistant also provides additional context and recommendations.

The Assistant can handle simple, descriptive, and conversational questions, enhancing the user experience in navigating Databricks' features and resolving issues. It can even help guide users in filing support tickets! For more details, read the announcement article.

CONCLUSION

The barrier to entry for quality data engineering has been lowered thanks to the power of generative AI with the Databricks Assistant. Whether you are a newcomer looking for help on how to work with complex data structures or a seasoned veteran who wants regular expressions written for them, the Assistant will improve your quality of life. Its core competency of understanding, generating, and documenting code boosts productivity for data engineers of all skill levels. To learn more, see the [Databricks documentation](#) on how to get started with the Databricks Assistant today.

Applying Software Development and DevOps Best Practices to Delta Live Table Pipelines

by Alex Ott

Databricks Delta Live Tables (DLT) radically simplifies the development of the robust data processing pipelines by decreasing the amount of code that data engineers need to write and maintain. And also reduces the need for data maintenance and infrastructure operations, while enabling users to seamlessly promote code and pipelines configurations between environments. But people still need to perform testing of the code in the pipelines, and we often get questions on how people can do it efficiently.

In this chapter we'll cover the following items based on our experience working with multiple customers:

- How to apply **DevOps** best practices to Delta Live Tables
- How to structure the DLT pipeline's code to facilitate unit and integration testing
- How to perform unit testing of individual transformations of your DLT pipeline
- How to perform integration testing by executing the full DLT pipeline
- How to promote the DLT assets between stages
- How to put everything together to form a CI/CD pipeline (with Azure DevOps as an example)



APPLYING DEVOPS PRACTICES TO DLT: THE BIG PICTURE

The DevOps practices are aimed at shortening the software development life cycle (SDLC) providing the high quality at the same time. Typically they include these steps:

- Version control of the source code and infrastructure
- Code reviews
- Separation of environments (development/staging/production)
- Automated testing of individual software components and the whole product with the unit and integration tests
- Continuous integration (testing) and continuous deployment of changes (CI/CD)

All these practices can be applied to Delta Live Tables pipelines as well:

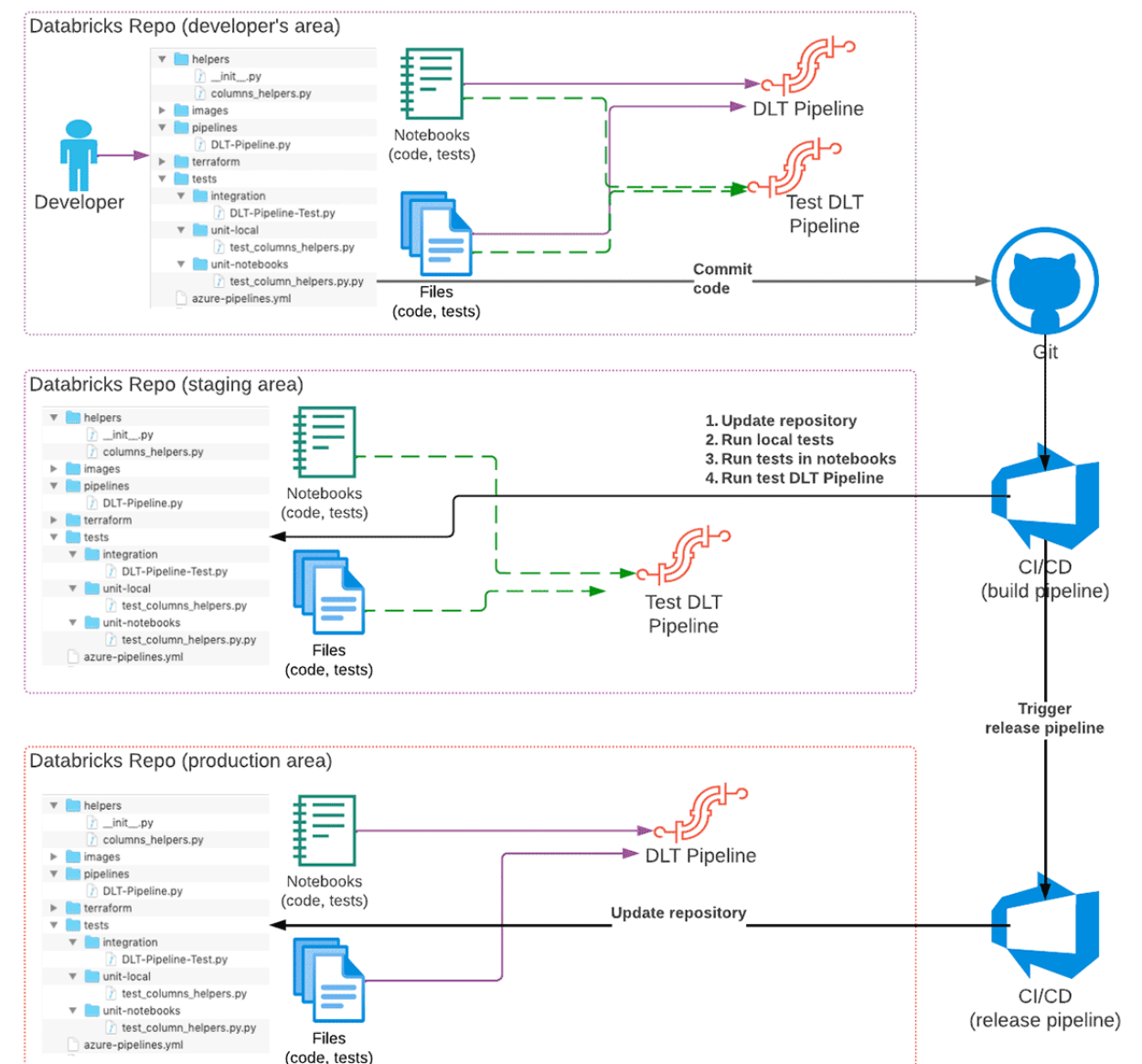
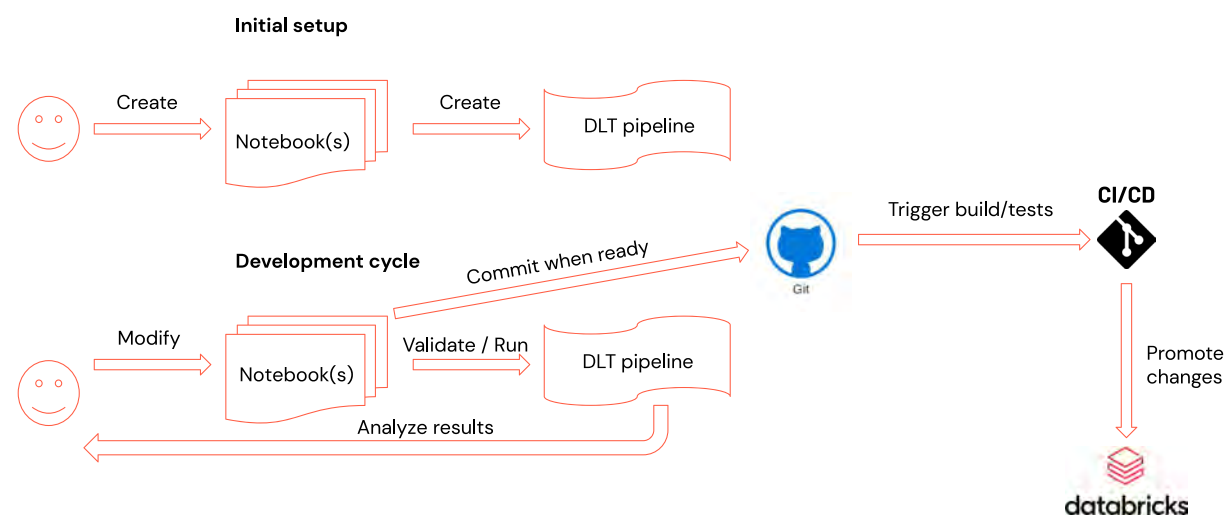


Figure: DLT development workflow

To achieve this we use the following features of Databricks product portfolio:

- **Databricks Repos** provide an interface to different Git services, so we can use them for code versioning, integration with CI/CD systems, and promotion of the code between environments
- **Databricks CLI** (or **Databricks REST API**) to implement CI/CD pipelines
- **Databricks Terraform Provider** for deployment of all necessary infrastructure and keeping it up to date

The recommended high-level development workflow of a DLT pipeline is as following:



1. A developer is developing the DLT code in their own checkout of a Git repository using a separate Git branch for changes.
2. When code is ready and tested, code is committed to Git and a pull request is created.

3. CI/CD system reacts to the commit and starts the build pipeline (CI part of CI/CD) that will update a staging Databricks Repo with the changes, and trigger execution of unit tests.

- a. Optionally, the integration tests could be executed as well, although in some cases this could be done only for some branches, or as a separate pipeline.

4. If all tests are successful and code is reviewed, the changes are merged into the main (or a dedicated branch) of the Git repository.

5. Merging of changes into a specific branch (for example, releases) may trigger a release pipeline (CD part of CI/CD) that will update the Databricks Repo in the production environment, so code changes will take effect when pipeline runs next time.

As illustration for the rest of the chapter we'll use a very simple DLT pipeline consisting just of two tables, illustrating typical **Bronze/Silver layers** of a typical **lakehouse architecture**. Complete source code together with deployment instructions is **available on GitHub**.



Figure: Example DLT pipeline

Note: DLT provides both SQL and Python APIs, in most of the chapter we focus on Python implementation, although we can apply most of the best practices also for SQL-based pipelines.

DEVELOPMENT CYCLE WITH DELTA LIVE TABLES

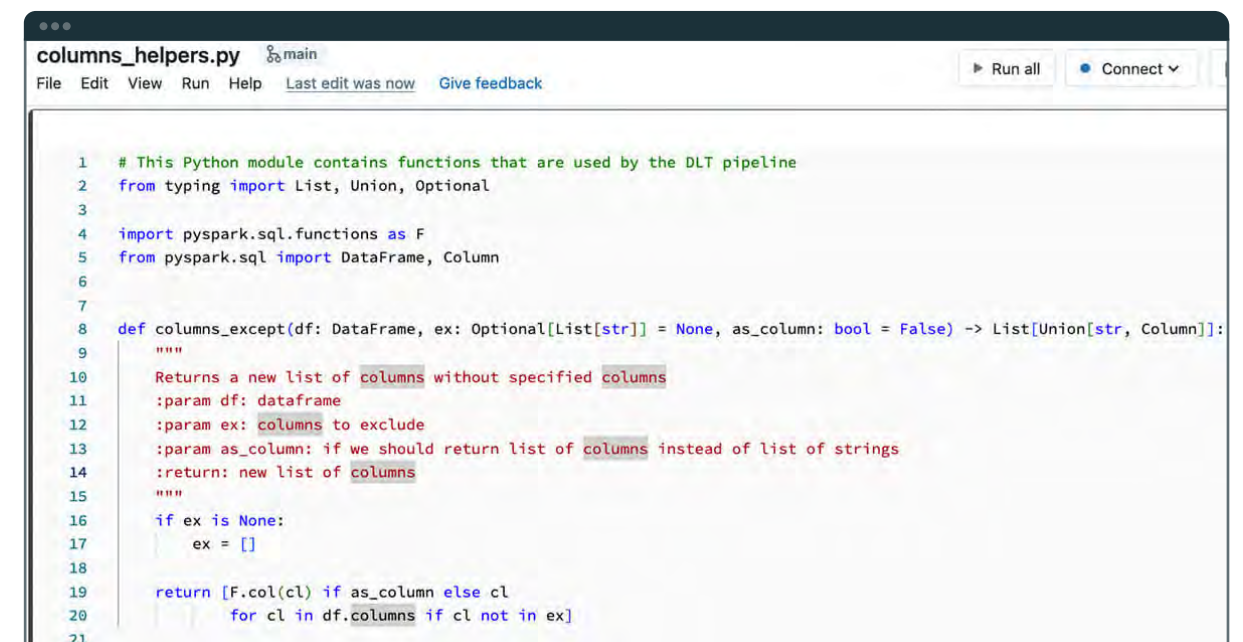
When developing with Delta Live Tables, typical development process looks as follows:

1. Code is written in the notebook(s).
2. When another piece of code is ready, a user switches to DLT UI and starts the pipeline. (To make this process faster it's recommended to run the pipeline in the **Development mode**, so you don't need to wait for resources again and again).
3. When a pipeline is finished or failed because of the errors, the user analyzes results, and adds/modifies the code, repeating the process.
4. When code is ready, it's committed.

For complex pipelines, such dev cycle could have a significant overhead because the pipeline's startup could be relatively long for complex pipelines with dozens of tables/views and when there are many libraries attached. For users it would be easier to get very fast feedback by evaluating the individual transformations and testing them with sample data on interactive clusters.

STRUCTURING THE DLT PIPELINE'S CODE

To be able to evaluate individual functions and make them testable it's very important to have correct code structure. Usual approach is to define all data transformations as individual functions receiving and returning Spark DataFrames, and call these functions from DLT pipeline functions that will form the DLT execution graph. The best way to achieve this is to use **files in repos** functionality that allows to expose Python files as normal Python modules that could be imported into Databricks notebooks or other Python code. DLT natively supports files in repos that allows **importing Python files as Python modules** (please note, that when using files in repos, the two entries are added to the Python's sys.path — one for repo root, and one for the current directory of the caller notebook). With this, we can start to write our code as a separate Python file located in the dedicated folder under the repo root that will be imported as a Python module:



```
columns_helpers.py  main
File Edit View Run Help Last edit was now Give feedback Run all Connect

1  # This Python module contains functions that are used by the DLT pipeline
2  from typing import List, Union, Optional
3
4  import pyspark.sql.functions as F
5  from pyspark.sql import DataFrame, Column
6
7
8  def columns_except(df: DataFrame, ex: Optional[List[str]] = None, as_column: bool = False) -> List[Union[str, Column]]:
9      """
10         Returns a new list of columns without specified columns
11         :param df: dataframe
12         :param ex: columns to exclude
13         :param as_column: if we should return list of columns instead of list of strings
14         :return: new list of columns
15         """
16         if ex is None:
17             ex = []
18
19         return [F.col(cl) if as_column else cl
20                 for cl in df.columns if cl not in ex]
21
```

Figure: Source code for a Python package

And the code from this Python package could be used inside the DLT pipeline code:

```

1 # import helper functions from the current repository
2 import helpers.columns_helpers as ch
3
4 @dlt.table(
5     comment="Leave only links, and exclude some columns from dataset"
6 )
7 @dlt.expect_or_drop("only links", "type is not null and type in ('link', 'redlink')")
8 def clickstream_filtered():
9     df = dlt.read("clickstream_raw")
10    # use imported function
11    new_cols = ch.columns_except(df, ['prev_id', 'prev_title'])
12    return df.select(*new_cols)
  
```

Figure: Using functions from the Python package in the DLT code

Note, that function in this particular DLT code snippet is very small — all it's doing is just reading data from the upstream table, and applying our transformation defined in the Python module. With this approach we can make DLT code simpler to understand and easier to test locally or using a separate notebook attached to an interactive cluster. Splitting the transformation logic into a separate Python module allows us to interactively test transformations from notebooks, write unit tests for these transformations and also test the whole pipeline (we'll talk about testing in the next sections).

The final layout of the Databricks Repo, with unit and integration tests, may look as following:

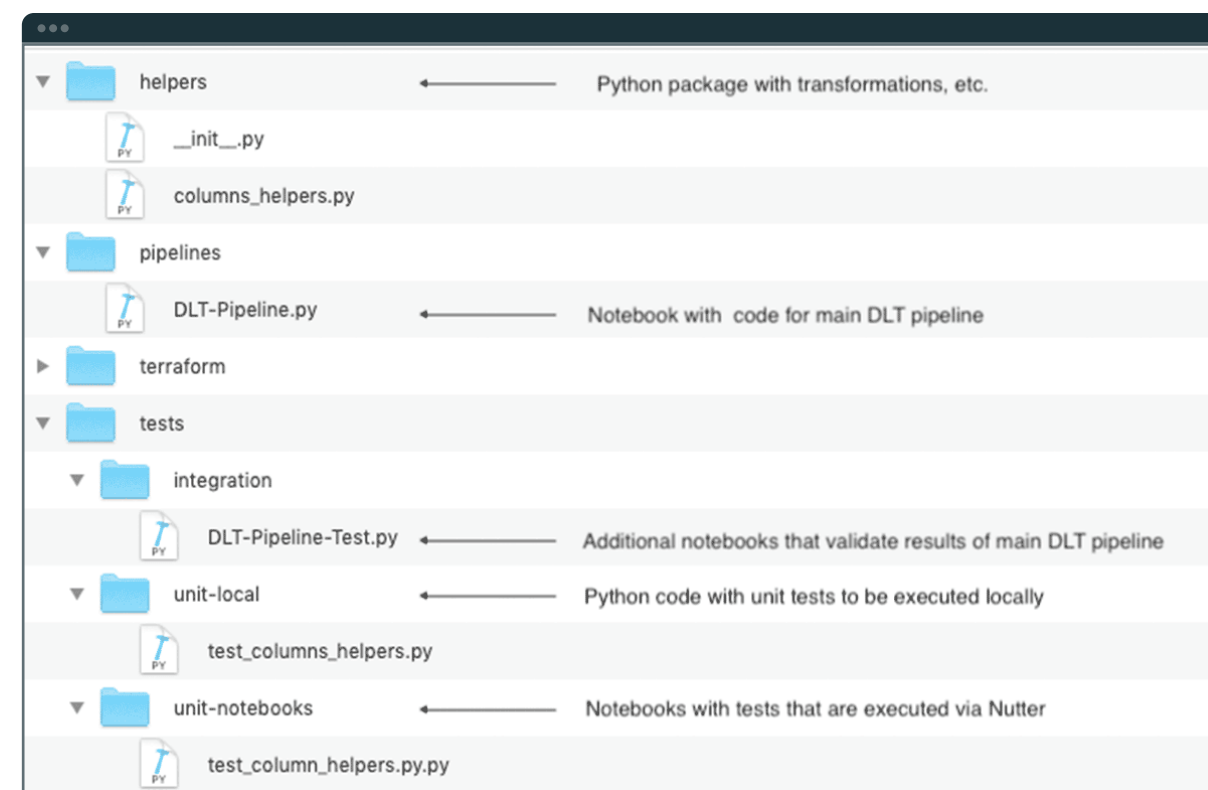


Figure: Recommended code layout in Databricks Repo

This code structure is especially important for bigger projects that may consist of the multiple DLT pipelines sharing the common transformations.

IMPLEMENTING UNIT TESTS

As mentioned above, splitting transformations into a separate Python module allows us easier write unit tests that will check behavior of the individual functions. We have a choice of how we can implement these unit tests:

- We can define them as Python files that could be executed locally, for example, using pytest. This approach has following advantages:
 - We can develop and test these transformations using the IDE, and for example, sync the local code with Databricks repo using the **Databricks extension for Visual Studio Code** or **dbx sync** command if you use another IDE
 - Such tests could be executed inside the CI/CD build pipeline without need to use Databricks resources (although it may depend if some Databricks-specific functionality is used or the code could be executed with PySpark)
 - We have access to more development related tools — static code and code coverage analysis, code refactoring tools, interactive debugging, etc.
 - We can even package our Python code as a library, and attach to multiple projects
- We can define them in the notebooks — with this approach:
 - We can get feedback faster as we always can run sample code and tests interactively
 - We can use additional tools like **Nutter** to trigger execution of notebooks from the CI/CD build pipeline (or from the local machine) and collect results for reporting

The demo repository contains a sample code for both of these approaches — for **local execution of the tests**, and **executing tests as notebooks**. The **CI pipeline** shows both approaches.

Please note that both of these approaches are applicable only to the Python code — if you're implementing your DLT pipelines using SQL, then you need to follow the approach described in the next section.

IMPLEMENTING INTEGRATION TESTS

While unit tests give us assurance that individual transformations are working as they should, we still need to make sure that the whole pipeline also works. Usually this is implemented as an integration test that runs the whole pipeline, but usually it's executed on the smaller amount of data, and we need to validate execution results. With Delta Live Tables, there are multiple ways to implement integration tests:

- Implement it as a Databricks Workflow with multiple tasks — similarly what is typically done for non-DLT code
- Use DLT expectations to check pipeline's results

IMPLEMENTING INTEGRATION TESTS WITH DATABRICKS WORKFLOWS

In this case we can implement integration tests with Databricks Workflows with multiple tasks (we can even pass data, such as, data location, etc. between tasks using **task values**). Typically such a workflow consists of the following tasks:

- Setup data for DLT pipeline
- Execute pipeline on this data
- Perform validation of produced results.

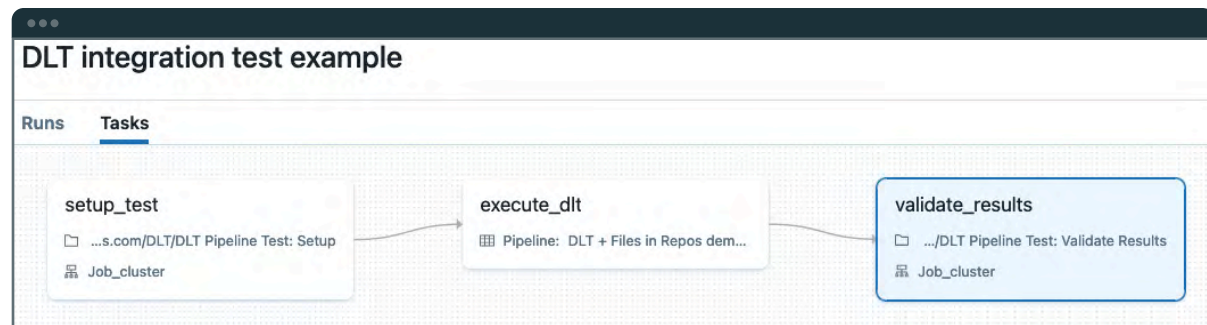


Figure: Implementing integration test with Databricks Workflows

The main drawback of this approach is that it requires writing quite a significant amount of the auxiliary code for setup and validation tasks, plus it requires additional compute resources to execute the setup and validation tasks.

USE DLT EXPECTATIONS TO IMPLEMENT INTEGRATION TESTS

We can implement integration tests for DLT by expanding the DLT pipeline with additional DLT tables that will apply **DLT expectations** to data **using the fail operator** to fail the pipeline if results don't match to provided expectations. It's very easy to implement – just create a separate DLT pipeline that will include additional notebook(s) that define DLT tables with expectations attached to them.

For example, to check that silver table includes only allowed data in the type column we can add following DLT table and attach expectations to it:

```

1 @dlt.table(comment="Check type")
2 @dlt.expect_all_or_fail({"valid type": "type in ('link', 'redlink')",
3                           "type is not null": "type is not null"})
4 def filtered_type_check():
5     return dlt.read("clickstream_filtered").select("type")
  
```

Resulting DLT pipeline for integration test may look as following (we have two additional tables in the execution graph that check that data is valid):

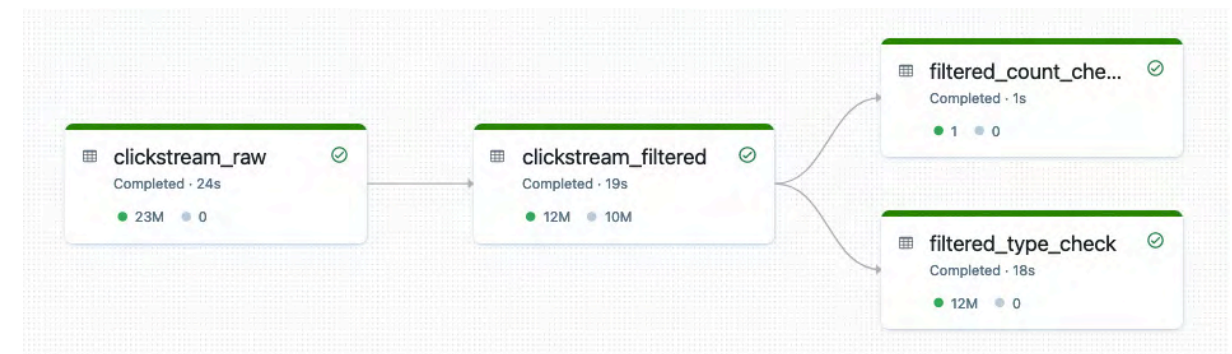


Figure: Implementing integration tests using DLT expectations

This is the recommended approach to performing integration testing of DLT pipelines. With this approach, we don't need any additional compute resources – everything is executed in the same DLT pipeline, so get cluster reuse, all data is logged into the **DLT pipeline's event log** that we can use for reporting, etc.

Please refer to DLT documentation for **more examples** of using DLT expectations for advanced validations, such as, checking uniqueness of rows, checking presence of specific rows in the results, etc. We can also build **libraries of DLT expectations** as shared Python modules for reuse between different DLT pipelines.

PROMOTING THE DLT ASSETS BETWEEN ENVIRONMENTS

When we're talking about promotion of changes in the context of DLT, we're talking about multiple assets:

- Source code that defines transformations in the pipeline
- Settings for a specific Delta Live Tables pipeline

The simplest way to promote the code is to use **Databricks Repos** to work with the code stored in the Git repository. Besides keeping your code versioned, Databricks Repos allows you to easily propagate the code changes to other environments using the **Repos REST API** or **Databricks CLI**.

From the beginning, DLT separates code from the **pipeline configuration** to make it easier to promote between stages by allowing to specify the schemas, data locations, etc. So we can define a separate DLT configuration for each stage that will use the same code, while allowing you to store data in different locations, use different cluster sizes, etc.

To define pipeline settings we can use **Delta Live Tables REST API** or **Databricks CLI's pipelines command**, but it becomes difficult in case you need to use instance pools, cluster policies, or other dependencies. In this case the more flexible alternative is Databricks Terraform Provider's **databricks_pipeline resource** that allows easier handling of dependencies to other resources, and we can use Terraform modules to modularize the Terraform code to make it reusable. The provided code repository **contains examples** of the Terraform code for deploying the DLT pipelines into the multiple environments.

PUTTING EVERYTHING TOGETHER TO FORM A CI/CD PIPELINE

After we implemented all the individual parts, it's relatively easy to implement a CI/CD pipeline. GitHub repository includes a **build pipeline for Azure DevOps** (other systems could be supported as well — the differences are usually in the file structure). This pipeline has two stages to show ability to execute different sets of tests depending on the specific event:

- onPush is executed on push to any Git branch except releases branch and version tags. This stage only runs and reports unit tests results (both local and notebooks).
- onRelease is executed only on commits to the releases branch, and in addition to the unit tests it will execute a DLT pipeline with integration test.

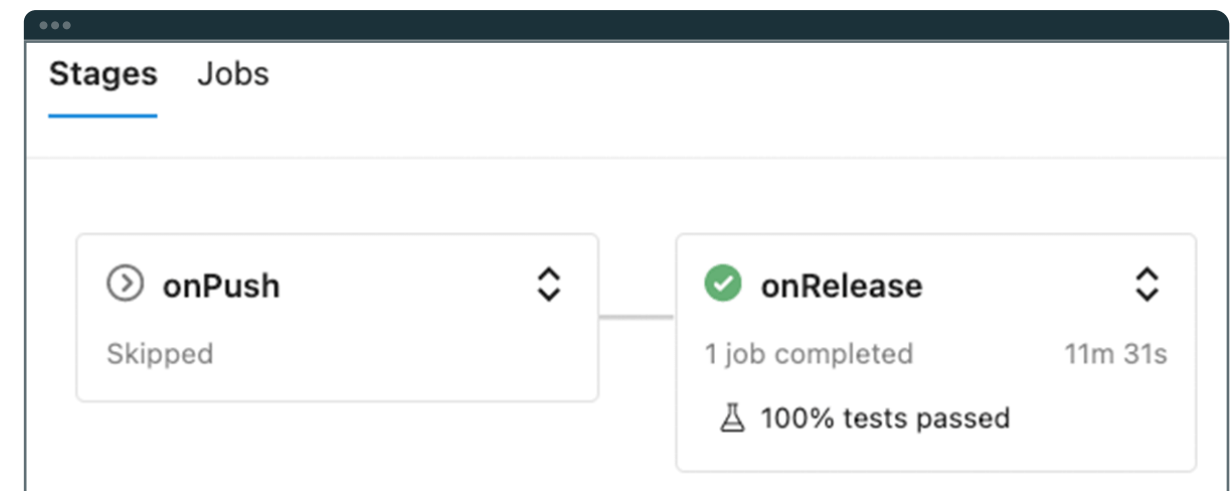


Figure: Structure of Azure DevOps build pipeline

Except for the execution of the integration test in the onRelease stage, the structure of both stages is the same — it consists of following steps:

1. Checkout the branch with changes
2. Set up environment — install **Poetry** which is used for managing Python environment management, and installation of required dependencies
3. Update Databricks Repos in the staging environment
4. Execute local unit tests using the PySpark
5. Execute the unit tests implemented as Databricks notebooks using Nutter
6. For **releases** branch, execute integration tests
7. Collect test results and publish them to Azure DevOps

Jobs in run #20221221.6
DLT Files In Repos Build Pipeline

onReleaseJob 11m 29s

- Initialize job 2s
- Use Python 3.8 <1s
- Checkout & Build.Rea... 1s
- Install dependen... 1m 20s
- Add poetry to PATH <1s
- Update Staging project 2s
- Execute local tests 12s
- Execute Nutter tests 22s
- Execute DLT Inte... 9m 16s
- PublishTestResults 4s
- Post-job: Checkout ... <1s

onReleaseJob

- Pool: Azure Pipelines
- Image: ubuntu-20.04
- Agent: Hosted Agent
- Started: Yesterday at 16:59
- Duration: 11m 29s
-
- Job preparation parameters
- 🔊 100% tests passed

Results of tests execution are reported back to the Azure DevOps, so we can track them:

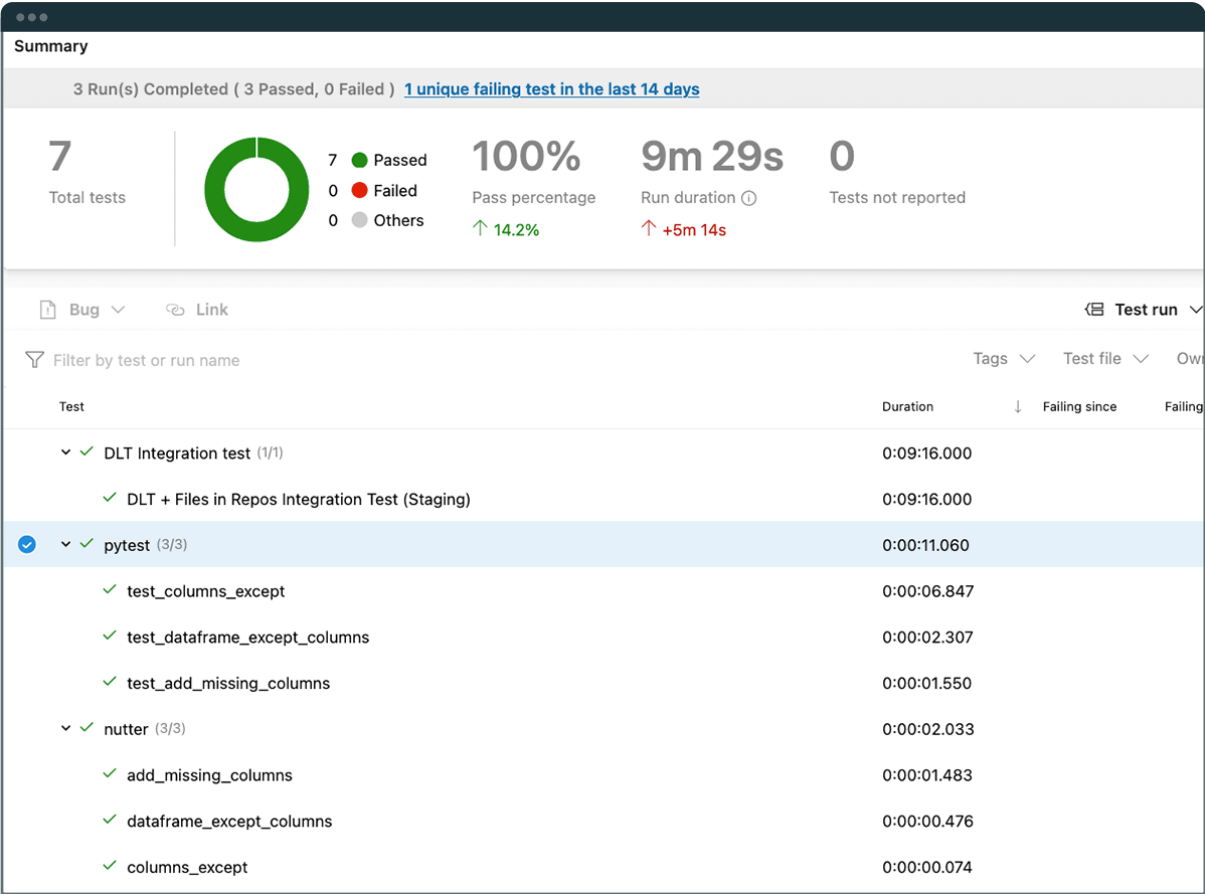


Figure: Reporting the tests execution results

If commits were done to the releases branch and all tests were successful, the **release pipeline** could be triggered, updating the production Databricks repo, so changes in the code will be taken into account on the next run of DLT pipeline.

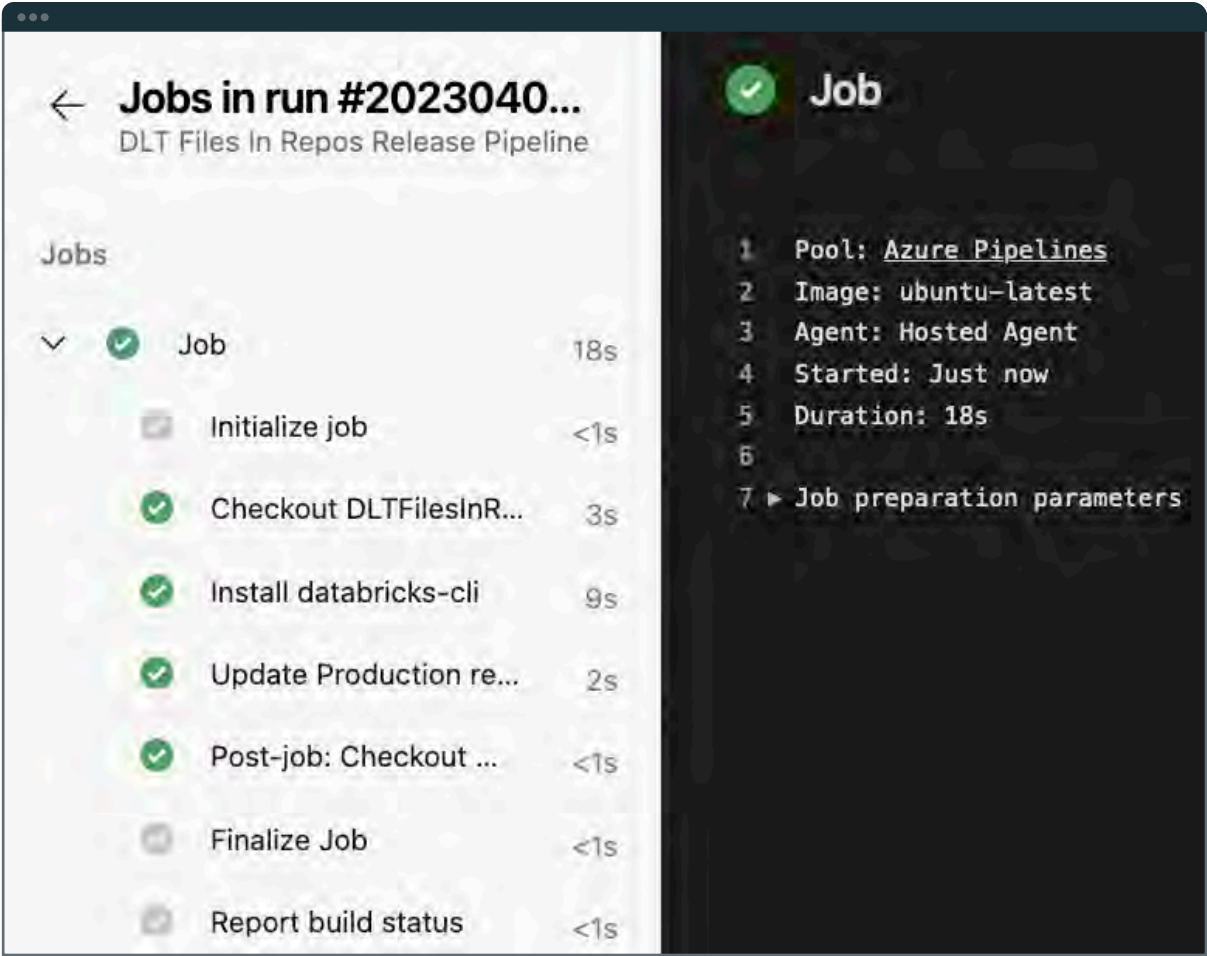


Figure: Release pipeline to deploy code changes to production DLT pipeline

Try to apply approaches described in this chapter to your Delta Live Table pipelines! The provided **demo repository** contains all necessary code together with setup instructions and Terraform code for deployment of everything to Azure DevOps.

Unity Catalog Governance in Action: Monitoring, Reporting and Lineage

by Ari Kaplan and Pearl Ubaru

Databricks Unity Catalog (UC) provides a single unified governance solution for all of a company's data and AI assets across clouds and data platforms. This chapter digs deeper into the prior **Unity Catalog Governance Value Levers blog** to show how the technology itself specifically enables positive business outcomes through comprehensive data and AI monitoring, reporting, and lineage.

OVERALL CHALLENGES WITH TRADITIONAL NON-UNIFIED GOVERNANCE

The **Unity Catalog Governance Value Levers blog** discussed the “why” of the organizational importance of governance for information security, access control, usage monitoring, enacting guardrails, and obtaining “single source of truth” insights from their data assets. These challenges compound as their company grows and without Databricks UC, traditional governance solutions no longer adequately meet their needs.

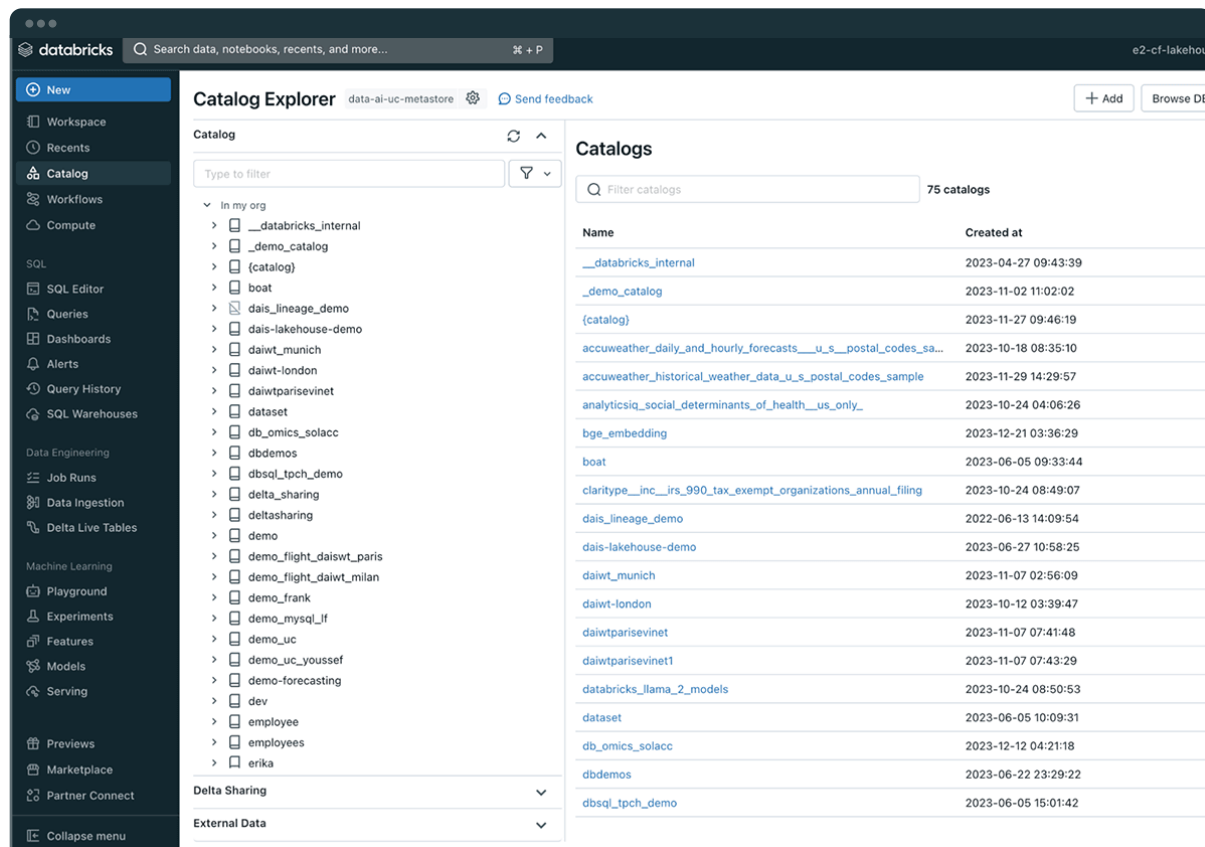
The major challenges discussed included weaker compliance and fractured data privacy controlled across multiple vendors; uncontrolled and siloed data and AI swamps; exponentially rising costs; loss of opportunities, revenue, and collaboration.

HOW DATABRICKS UNITY CATALOG SUPPORTS A UNIFIED VIEW, MONITORING, AND OBSERVABILITY

So, how does this all work from a technical standpoint? UC manages all registered assets across the Databricks Data Intelligence Platform. These assets can be anything within BI, DW, data engineering, data streaming, data science, and ML. This governance model provides access controls, lineage, discovery, monitoring, auditing, and sharing. It also provides metadata management of files, tables, ML models, notebooks, and dashboards. UC gives one single view of your entire end-to-end information, through the Databricks asset catalog, feature store and model registry, lineage capabilities, and metadata tagging for data classifications, as discussed below:

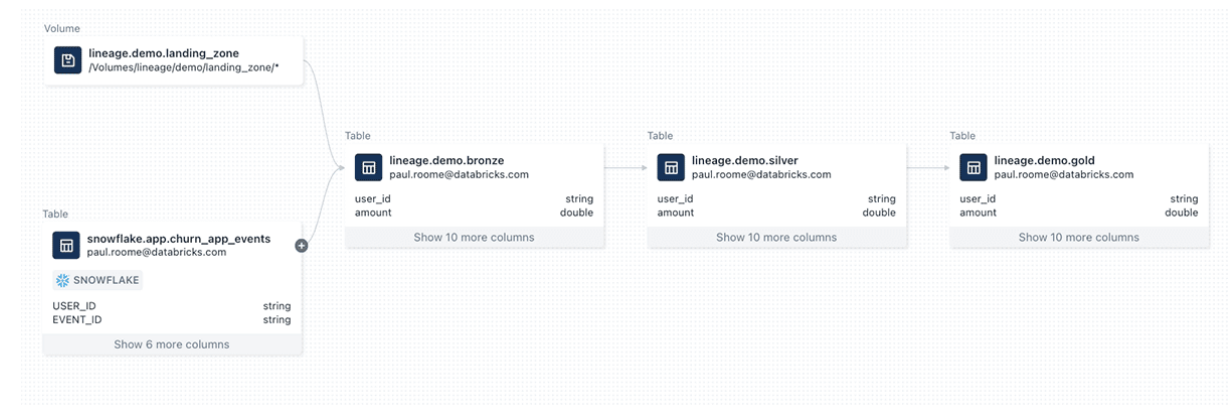
Unified view of the entire data estate

- **Asset catalog:** through system tables that contain metadata, you can see all that is contained in your catalog such as schemas, tables, columns, files, models, and more. If you are not familiar with volumes within Databricks, they are used for managing non-tabular datasets. Technically, they are logical volumes of storage to access files in any format: structured, semi-structured, and unstructured.



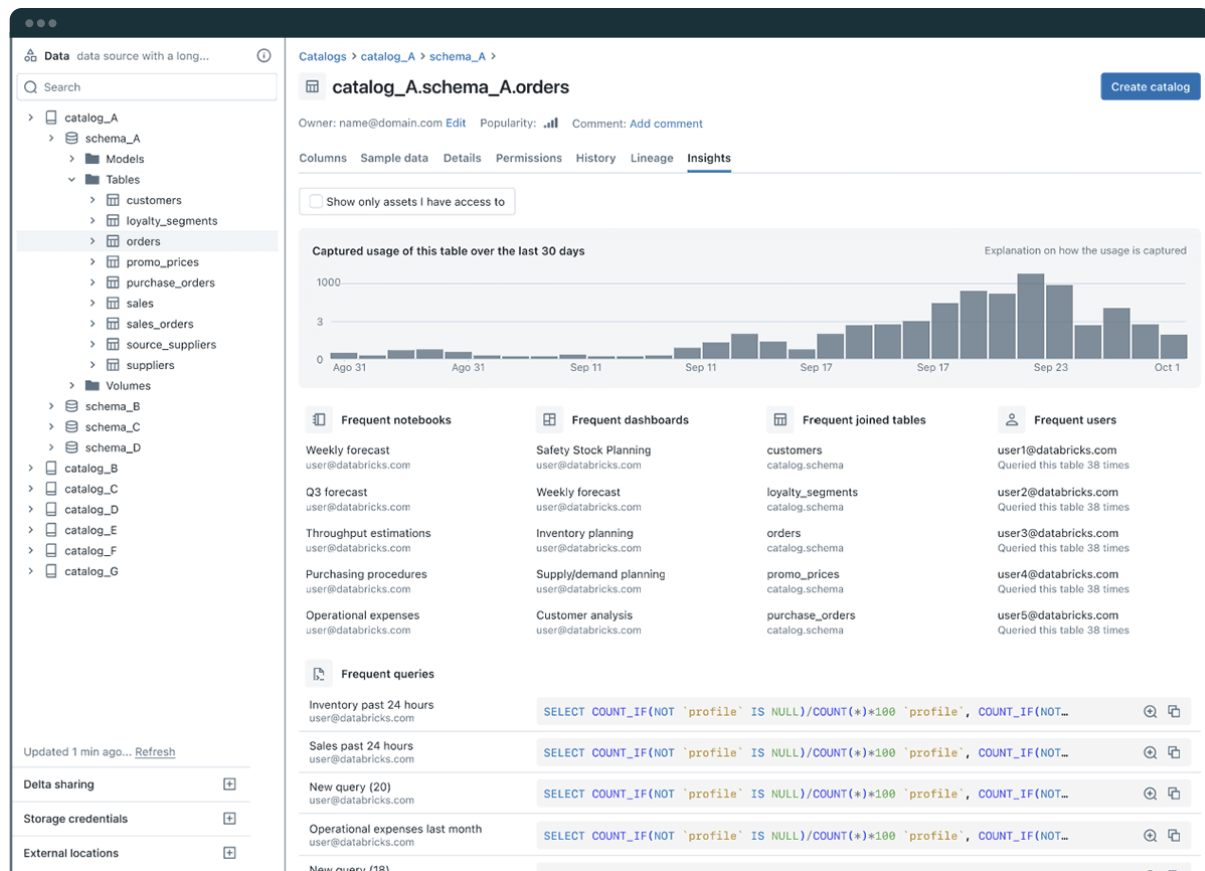
Catalog Explorer lets you discover and govern all your data and ML models

- **Feature Store** and **Model Registry:** define features used by data scientists within the centralized repository. This is helpful for consistent model training and inference for your entire AI workflow.
- **Lineage capabilities:** trust in your data is key for your business to take action in real life. End-to-end transparency into your data is needed for trust in your reports, models, and insights. UC makes this easy through lineage capabilities, providing insights on: What are the raw data sources? Who created it and when? How was data merged and transformed? What is the traceability from the models back to the datasets they are trained on? Lineage shows end-to-end from data to model – both table-level and column-level. You can even query across data sources such as Snowflake and benefit immediately:

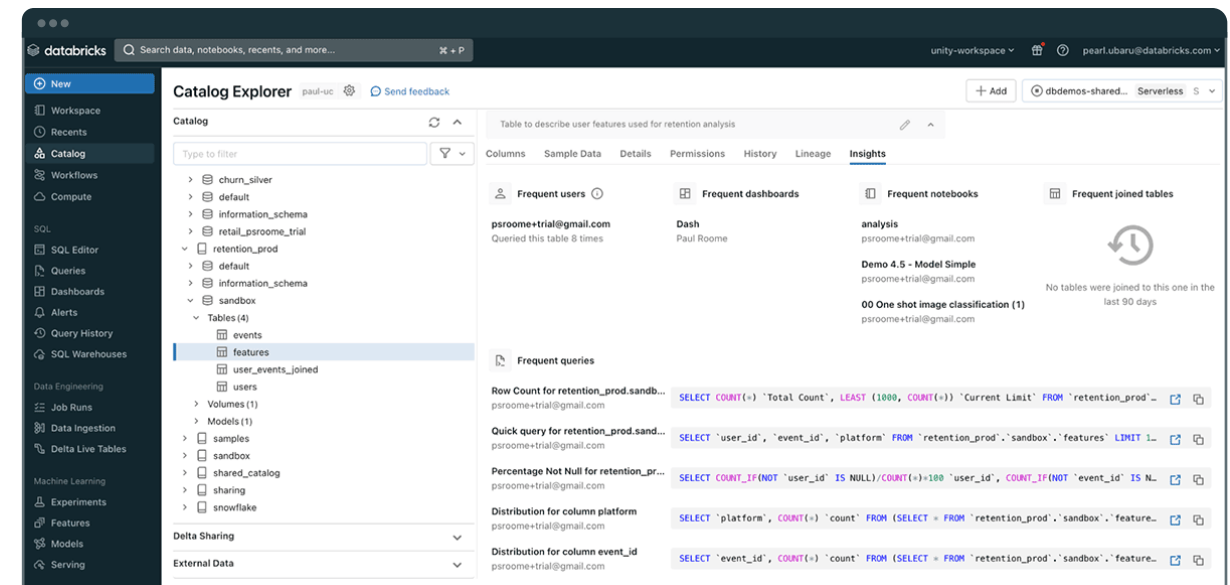


Data sources can be across platforms such as Snowflake and Databricks

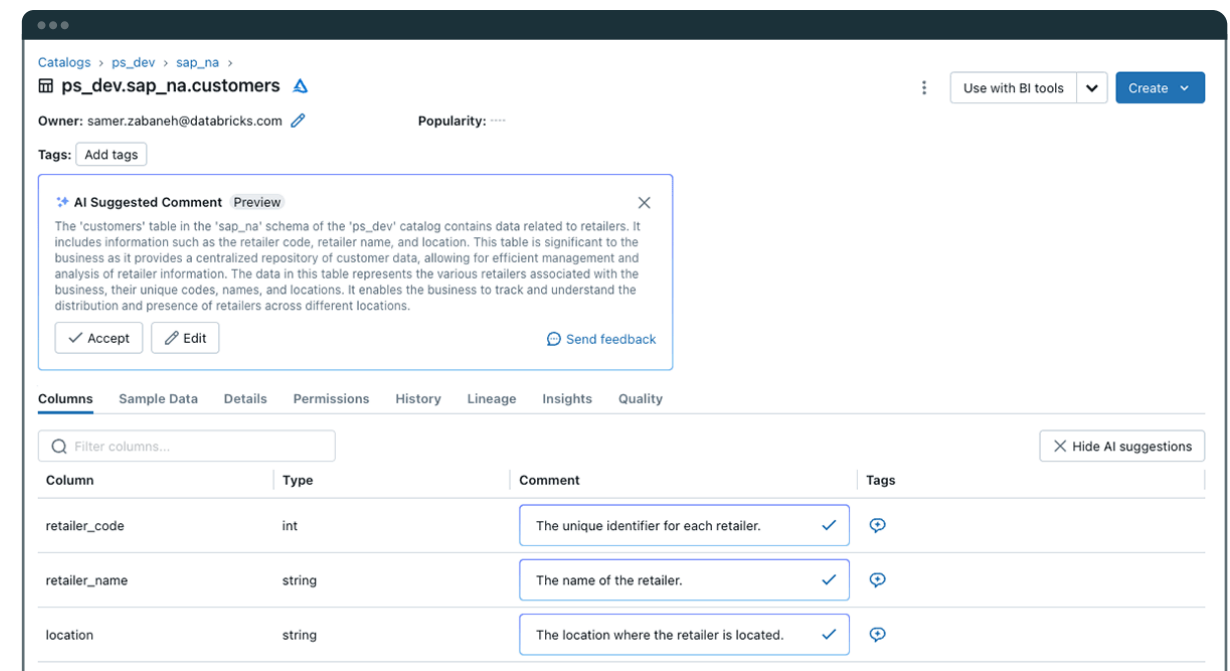
- **Metadata tagging** for data classifications: enrich your data and queries by providing contextual insights about your data assets. These descriptions at the column and table level can be manually entered, or automatically described with GenAI by Databricks Assistant. Below is an example of descriptions and quantifiable characteristics:



Metadata tagging insights: frequent users, notebooks, queries, joins, billing trends and more



Metadata tagging insights: details on the “features” table



Databricks Assistant uses GenAI to write context-aware descriptions of columns and tables

Having one unified view results in:

- **Accelerated innovation:** your insights are only as good as your data. Your analysis is only as good as the data you access. So, streamlining your data search drives faster and better generation of business insights, driving innovation.
- **Cost reduction through centralized asset cataloging:** lowers license costs (just one vendor solution versus needing many vendors), lowers usage fees, reduces time to market pains, and enables overall operational efficiencies.
- **It's easier to discover and access all data** by reducing data sprawl across several databases, data warehouses, object storage systems, and more.

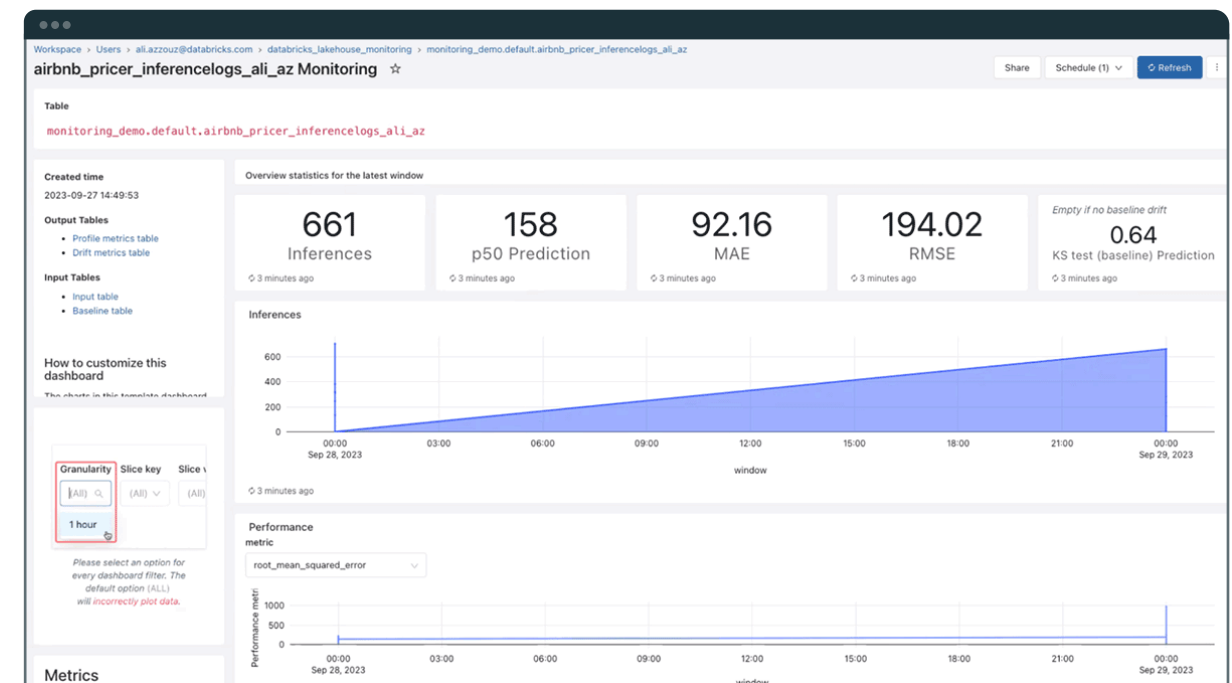
COMPREHENSIVE DATA AND AI MONITORING AND REPORTING

Databricks **Lakehouse Monitoring** allows teams to monitor their entire data pipelines — from data and features to ML models — without additional tools and complexity. Powered by Unity Catalog, it lets users uniquely ensure that their data and AI assets are high quality, accurate and reliable through deep insight into the lineage of their data and AI assets. The single, unified approach to monitoring enabled by lakehouse architecture makes it simple to diagnose errors, perform root cause analysis, and find solutions.

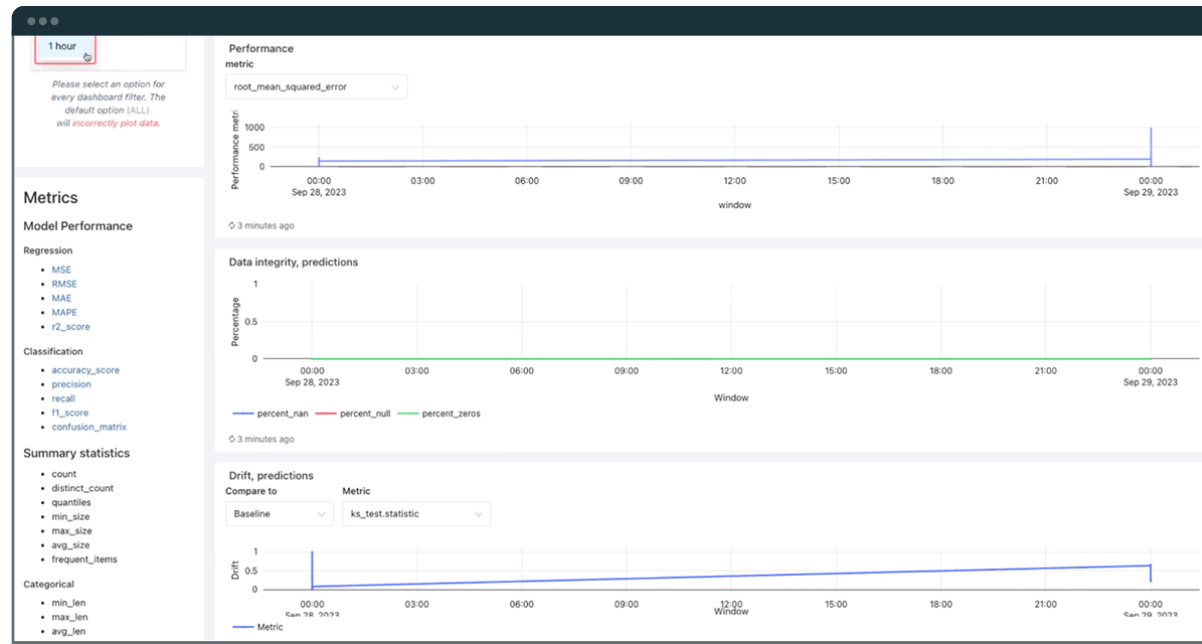
How do you ensure trust in your data, ML models, and AI across your entire data pipeline in a single view regardless of where the data resides? Databricks Lakehouse Monitoring is the industry's only comprehensive solution from data (regardless of where it resides) to insights. It accelerates the discovery of issues, helps determine root causes, and ultimately assists in recommending solutions.

UC provides Lakehouse Monitoring capabilities with both democratized dashboards and granular governance information that can be directly queried through system tables. The democratization of governance extends operational oversight and compliance to non-technical people, allowing a broad variety of teams to monitor all of their pipelines.

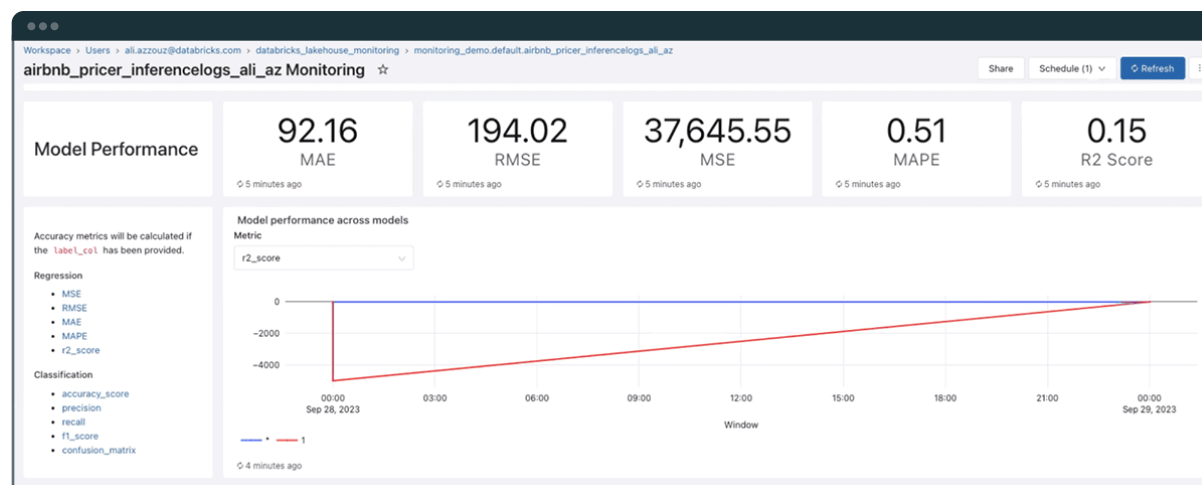
Below is a sample dashboard of the results of an ML model including its accuracy over time:



It further shows data integrity of predictions and data drift over time:

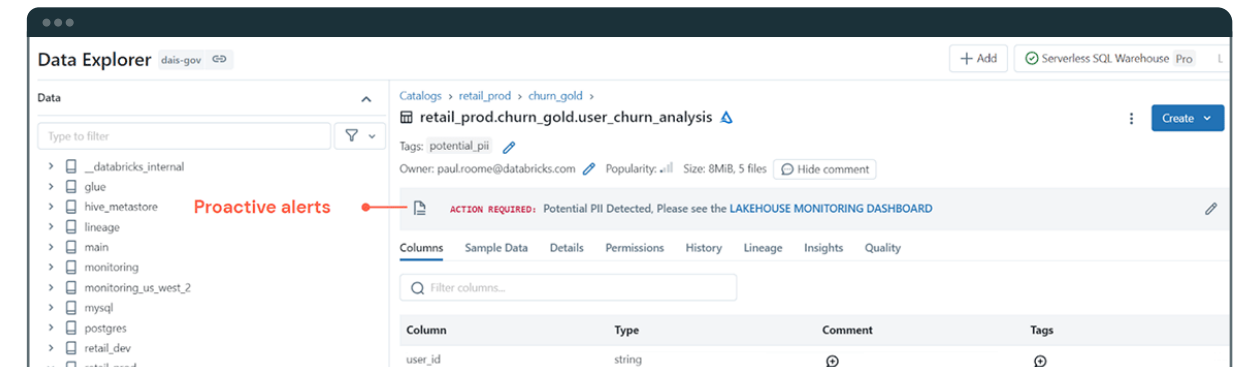


And model performance over time, according to a variety of ML metrics such as R2, RMSE, and MAPE:



Lakehouse Monitoring dashboards show data and AI assets quality

It's one thing to intentionally seek out ML model information when you are looking for answers, but it is a whole other level to get automated proactive alerts on errors, data drift, model failures, or quality issues. Below is an example alert for a potential PII (Personal Identifiable Information) data breach:



Example proactive alert of potential unmasked private data

One more thing — you can assess the impact of issues, do a root cause analysis, and assess the downstream impact by Databricks's powerful lineage capabilities — from table-level to column-level.

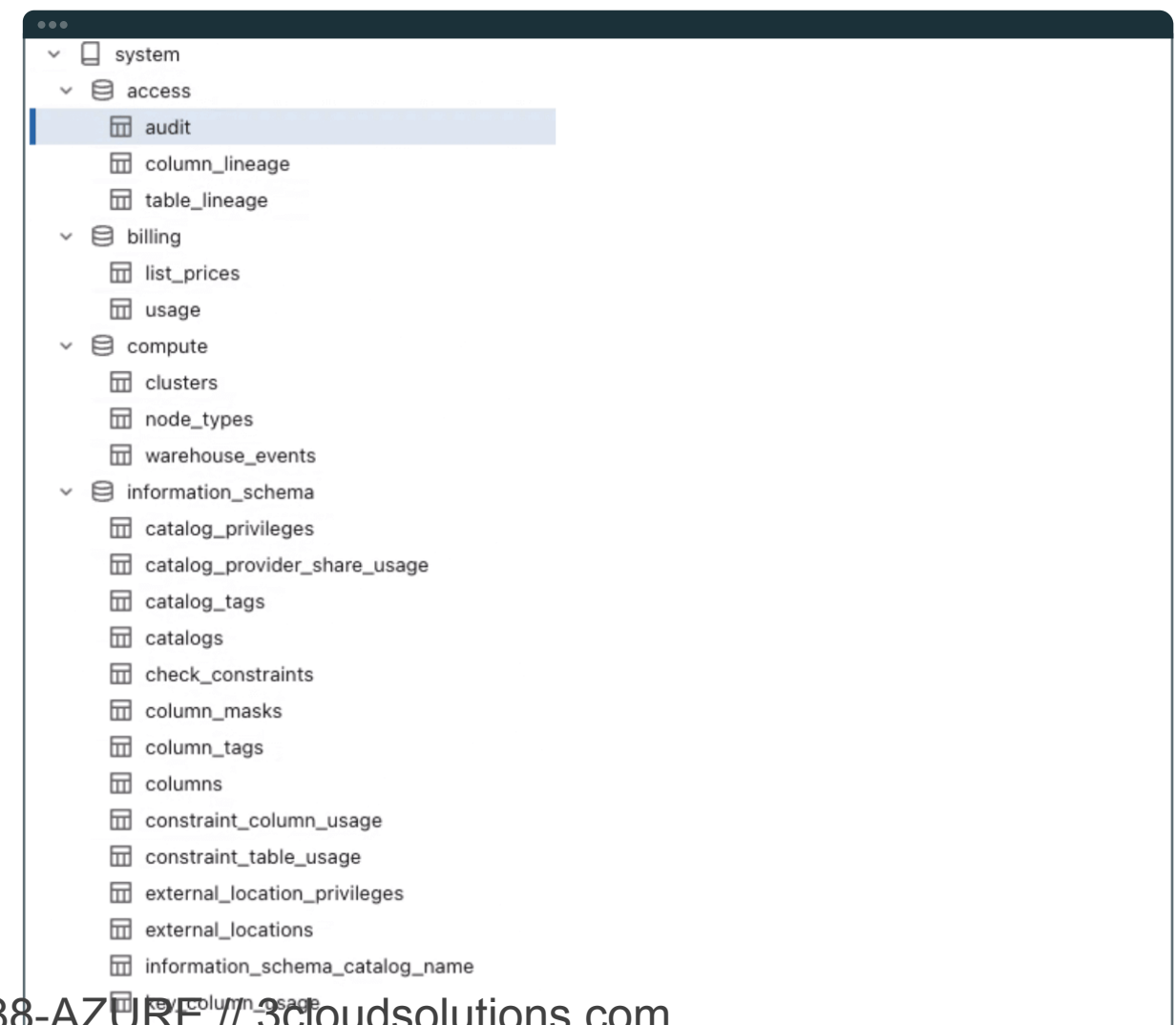
System tables: metadata information for lakehouse observability and ensuring compliance

These underlying tables can be queried through SQL or activity dashboards to provide observability about every asset within the Databricks Intelligence Platform. Examples include which users have access to which data objects; billing tables that provide pricing and usage; compute tables that take cluster usage and warehouse events into consideration; and lineage information between columns and tables:

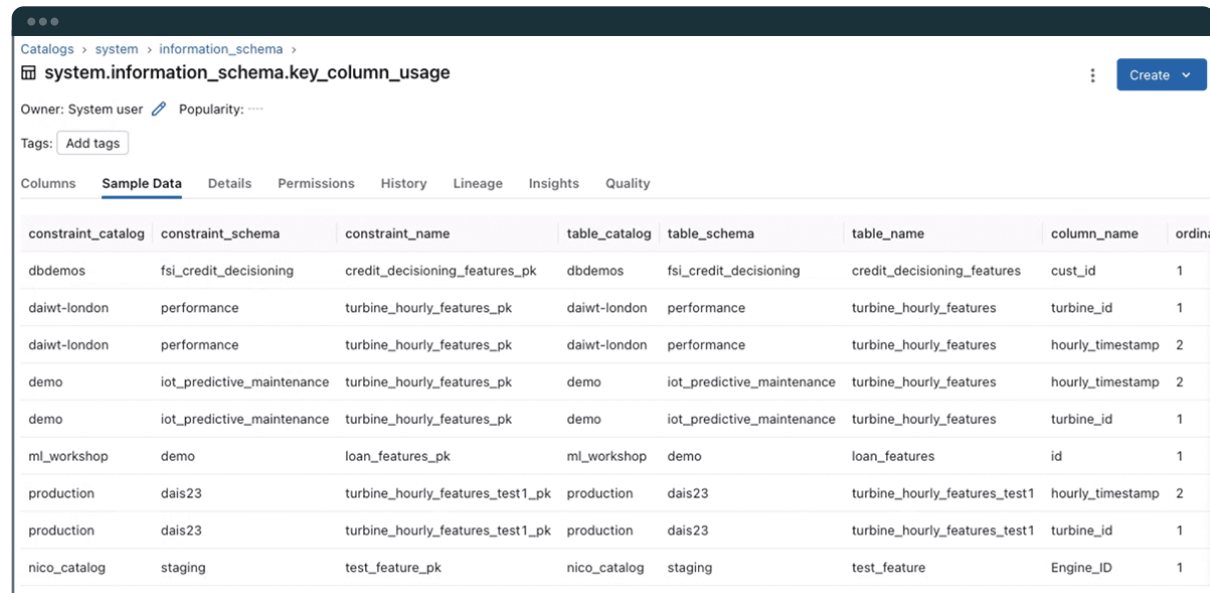
- Audit tables include information on a wide variety of UC events. UC captures an **audit log of actions** performed against the metastore giving administrators access to details about who accessed a given dataset and the actions that they performed.
- Billing and historical pricing tables will include records for all billable usage across the entire account; therefore you can view your account's global usage from whichever region your workspace is in.
- Table lineage and column lineage tables are great because they allow you to programmatically query lineage data to fuel decision making and reports. Table lineage records each read-and-write event on a UC table or path that might include job runs, notebook runs and dashboards associated with the table. For column lineage, data is captured by reading the column.
- Node types tables capture the currently available node types with their basic hardware information outlining the node type name, the number of vCPUs for the instance, and the number of GPUs and memory for the instance. Also in private preview are node_utilization metrics on how much usage each node is leveraging.
- Query history holds information on all SQL commands, i/o performance, and number of rows returned.

- Clusters table contains the full history of cluster configurations over time for all-purpose and job clusters.
- Predictive optimization tables are great because they optimize your data layout for peak performance and cost efficiency. The tables track the operation history of optimized tables by providing the catalog name, schema name, table name, and operation metrics about compaction and vacuuming.

From the catalog explorer, here are just a few of the system tables any of which can be viewed for more details:



As an example, drilling down on the "key_column_usage" table, you can see precisely how tables relate to each other via their primary key:



Catalogs > system > information_schema > **system.information_schema.key_column_usage**

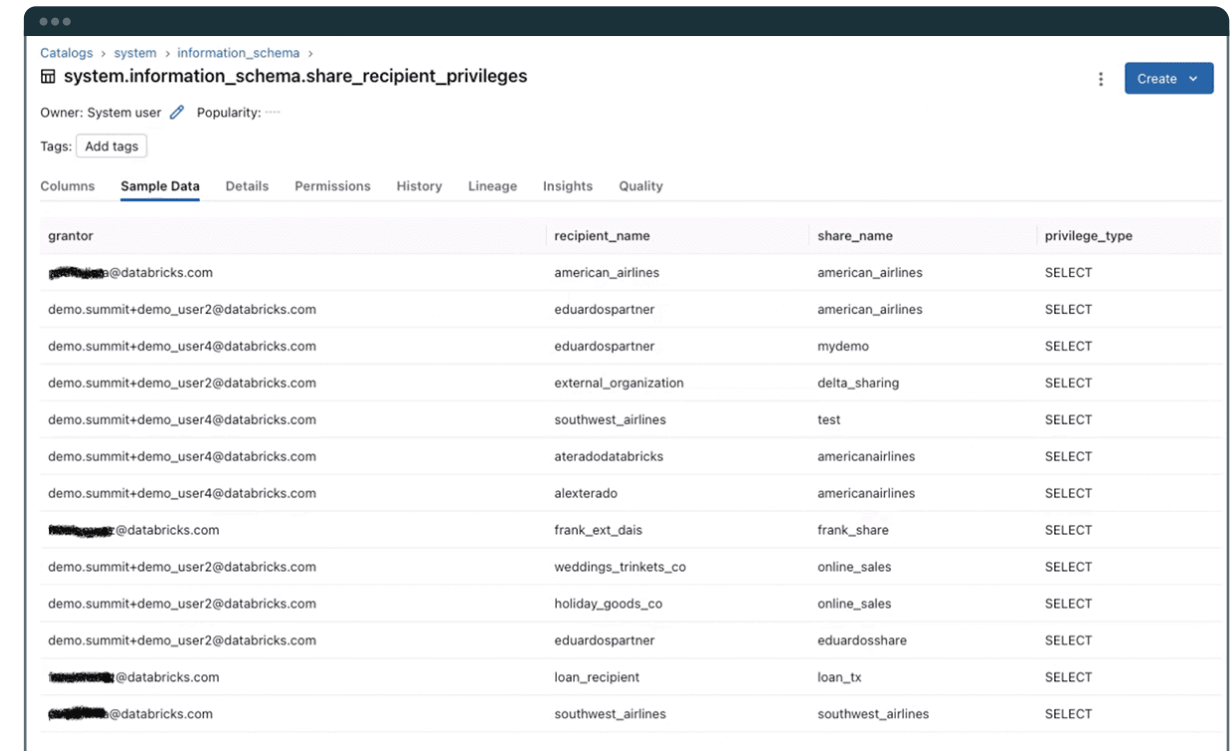
Owner: System user Popularity: ---

Tags: Add tags

Columns Sample Data Details Permissions History Lineage Insights Quality

constraint_catalog	constraint_schema	constraint_name	table_catalog	table_schema	table_name	column_name	ordinal_position
dbdemos	fsi_credit_decisioning	credit_decisioning_features_pk	dbdemos	fsi_credit_decisioning	credit_decisioning_features	cust_id	1
daiwt-london	performance	turbine_hourly_features_pk	daiwt-london	performance	turbine_hourly_features	turbine_id	1
daiwt-london	performance	turbine_hourly_features_pk	daiwt-london	performance	turbine_hourly_features	hourly_timestamp	2
demo	iot_predictive_maintenance	turbine_hourly_features_pk	demo	iot_predictive_maintenance	turbine_hourly_features	hourly_timestamp	2
demo	iot_predictive_maintenance	turbine_hourly_features_pk	demo	iot_predictive_maintenance	turbine_hourly_features	turbine_id	1
ml_workshop	demo	loan_features_pk	ml_workshop	demo	loan_features	id	1
production	dais23	turbine_hourly_features_test1_pk	production	dais23	turbine_hourly_features_test1	hourly_timestamp	2
production	dais23	turbine_hourly_features_test1_pk	production	dais23	turbine_hourly_features_test1	turbine_id	1
nico_catalog	staging	test_feature_pk	nico_catalog	staging	test_feature	Engine_ID	1

Another example is the "share_recipient_privileges" table, to see who granted which shares to whom:



Catalogs > system > information_schema > **system.information_schema.share_recipient_privileges**

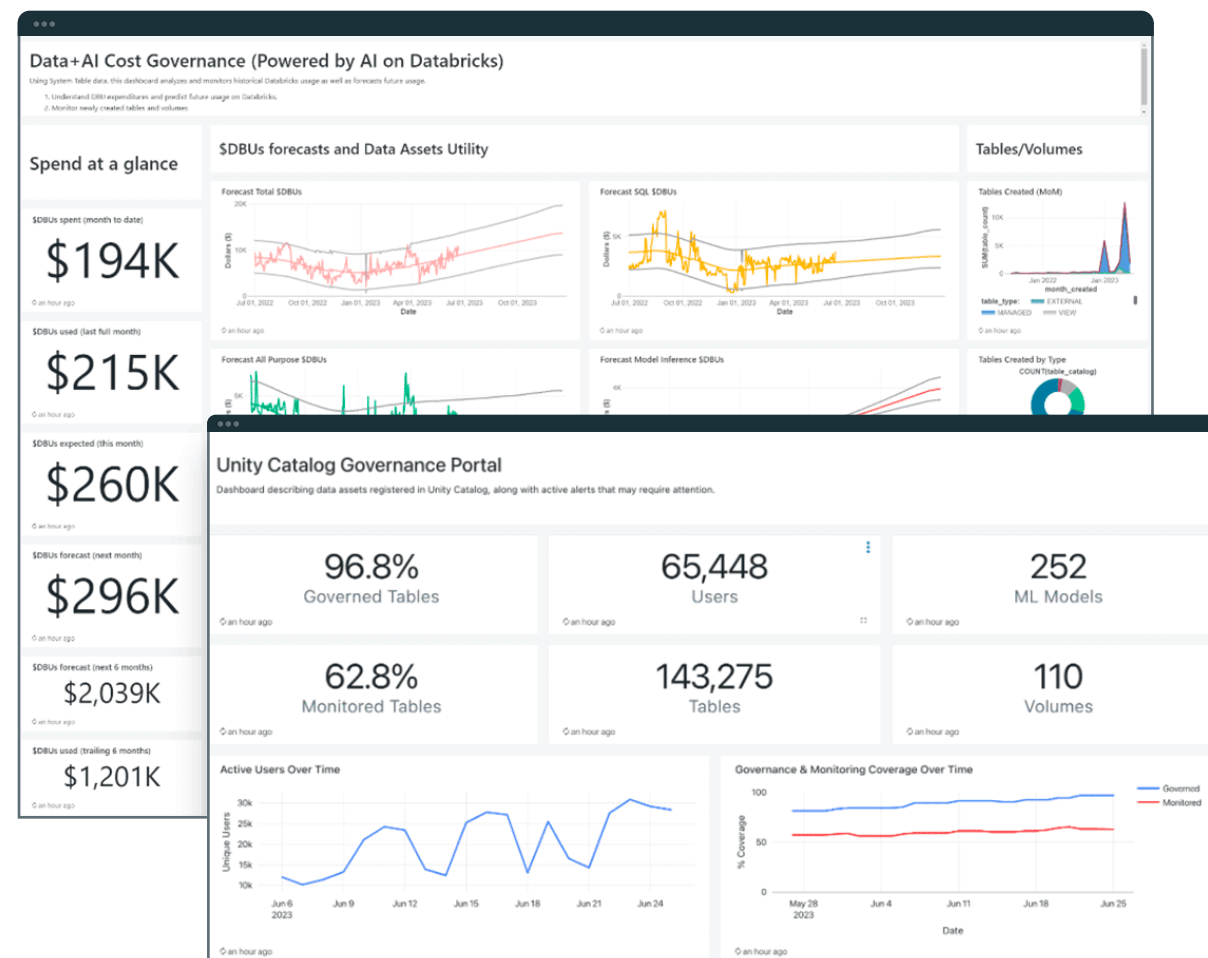
Owner: System user Popularity: ---

Tags: Add tags

Columns Sample Data Details Permissions History Lineage Insights Quality

grantor	recipient_name	share_name	privilege_type
[REDACTED]@databricks.com	american_airlines	american_airlines	SELECT
demo.summit+demo_user2@databricks.com	eduardospartner	american_airlines	SELECT
demo.summit+demo_user4@databricks.com	eduardospartner	mydemo	SELECT
demo.summit+demo_user2@databricks.com	external_organization	delta_sharing	SELECT
demo.summit+demo_user4@databricks.com	southwest_airlines	test	SELECT
demo.summit+demo_user4@databricks.com	ateradodatabricks	americanairlines	SELECT
demo.summit+demo_user4@databricks.com	alexterado	americanairlines	SELECT
[REDACTED]@databricks.com	frank_ext_dais	frank_share	SELECT
demo.summit+demo_user2@databricks.com	weddings_trinkets_co	online_sales	SELECT
demo.summit+demo_user2@databricks.com	holiday_goods_co	online_sales	SELECT
demo.summit+demo_user2@databricks.com	eduardospartner	eduardosshare	SELECT
[REDACTED]@databricks.com	loan_recipient	loan_tx	SELECT
[REDACTED]@databricks.com	southwest_airlines	southwest_airlines	SELECT

The example dashboard below shows the number of users, tables, ML models, percent of tables that are monitored or not, dollars spent on Databricks DBUs over time, and so much more:



Governance dashboard showing billing trends, usage, activity and more

What does having a comprehensive data and AI monitoring and reporting tool result in?

- Reduced risk of non-compliance with better monitoring of internal policies and security breach potential results in safeguarded reputation and improved data and AI trust from employees and partners.
- Improved integrity and trustworthiness of data and AI with "one source of truth", anomaly detection, and reliability metrics.

Value Levers with Databricks Unity Catalog

If you are looking to learn more about the values Unity Catalog brings to businesses, the prior [Unity Catalog Governance Value Levers](#) blog went into detail: mitigating risk around compliance; reducing platform complexity and costs; accelerating innovation; facilitating better internal and external collaboration; and monetizing the value of data.

CONCLUSION

Governance is key to mitigating risks, ensuring compliance, accelerating innovation, and reducing costs. Databricks Unity Catalog is unique in the market, providing a single unified governance solution for all of a company's data and AI across clouds and data platforms.

UC Databricks architecture makes governance seamless: a unified view and discovery of all data assets, one tool for access management, one tool for auditing for enhanced data and AI security, and ultimately enabling platform-independent collaboration that unlocks new business values.

Getting started is easy – UC comes enabled by default with Databricks if you are a new customer! Also if you are on premium or enterprise workspaces, there are no additional costs.

Scalable Spark Structured Streaming for REST API Destinations

by Art Rask and Jay Palaniappan

Spark Structured Streaming is the widely-used open source engine at the foundation of **data streaming on the Data Intelligence Platform**. It can elegantly handle diverse logical processing at volumes ranging from small-scale ETL to the largest Internet services. This power has led to adoption in many use cases across industries.

Another strength of Structured Streaming is its ability to handle a variety of both sources and sinks (or destinations). In addition to numerous sink types supported natively (incl. Delta, AWS S3, Google GCS, Azure ADLS, Kafka topics, Kinesis streams, and more), Structured Streaming supports a specialized sink that has the ability to perform arbitrary logic on the output of a streaming query: the `foreachBatch` extension method. With **`foreachBatch`**, any output target addressable through Python or Scala code can be the destination for a stream.

In this chapter we will share best practice guidance we've given customers who have asked how they can scalably turn streaming data into calls against a REST API. Routing an incoming stream of data to calls on a REST API is a requirement seen in many integration and data engineering scenarios.

Some practical examples that we often come across are in Operational and Security Analytics workloads. Customers want to ingest and enrich real-time streaming data from sources like kafka, eventhub, and Kinesis and publish it into operational search engines like Elasticsearch, Opensearch, and Splunk. A key advantage of Spark Streaming is that it allows us to enrich, perform data quality checks, and aggregate (if needed) before data is streamed out into the search engines. This provides customers a high quality real-time data pipeline for operational and security analytics.

The most basic representation of this scenario is shown in Figure 1. Here we have an incoming stream of data – it could be a Kafka topic, AWS Kinesis, Azure Event Hub, or any other streaming query source. As messages flow off the stream we need to make calls to a REST API with some or all of the message data.

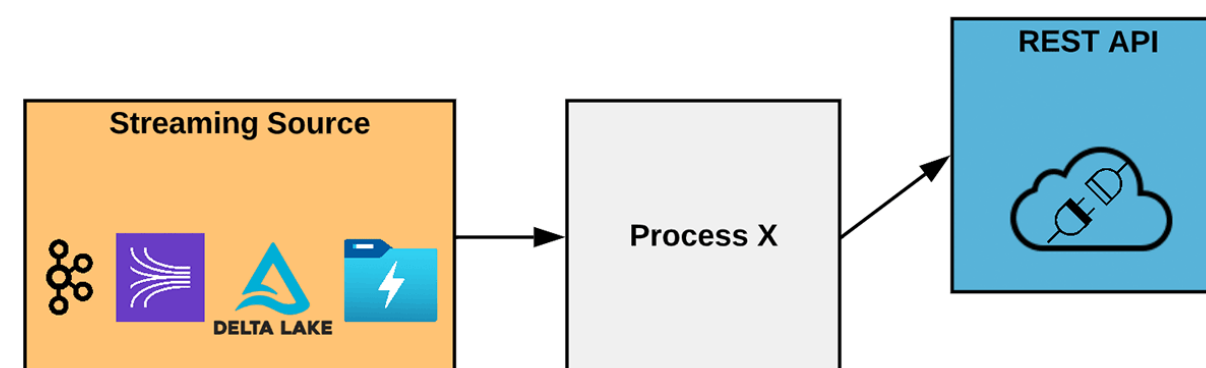


Figure 1

In a greenfield environment, there are many technical options to implement this. Our focus here is on teams that already have streaming pipelines in Spark for preparing data for machine learning, data warehousing, or other analytics-focused uses. In this case, the team will already have skills, tooling and DevOps processes for Spark. Assume the team now has a requirement to route some data to REST API calls. If they wish to leverage existing skills or avoid re-working their tool chains, they can use Structured Streaming to get it done.

KEY IMPLEMENTATION TECHNIQUES, AND SOME CODE

A basic code sample is included as Exhibit 1. Before looking at it in detail, we will call out some key techniques for effective implementation.

For a start, you will read the incoming stream as you would any other streaming job. All the interesting parts here are on the output side of the stream. If your data must be transformed in flight before posting to the REST API, do that as you would in any other case. This code snippet reads from a Delta table; as mentioned, there are many other possible sources.

```
1 dfSource = (spark.readStream
2               .format("delta")
3               .table("samples.nyctaxi.trips"))
```

For directing streamed data to the REST API, take the following approach:

1. Use the `foreachBatch` extension method to pass incoming micro-batches to a handler method (`callRestAPIBatch`) which will handle calls to the REST API.

```
1 streamHandle = (dfSource.writeStream
2                   .foreachBatch(callRestAPIBatch)
3                   .start())
```

2. Whenever possible, group multiple rows from the input on each outgoing REST API call. In relative terms, making the API call over HTTP will be a slow part of the process. Your ability to reach high throughput will be dramatically improved if you include multiple messages/records on the body of each API call. Of course, what you can do will be dictated by the target REST API. Some APIs allow a POST body to include many items up to a maximum body size in bytes. Some APIs have a max count of items on the POST body. Determine the max you can fit on a single call for the target API. In your method invoked by `foreachBatch`, you will have a prep step to transform the micro-batch dataframe into a pre-batched dataframe where each row has the grouped records for one call to the API. This step is also a chance for any last transform of the records to the format expected by the target API. An example is shown in the code sample in Exhibit 1 with the call to a helper function named `preBatchRecordsForRestCall`.
3. In most cases, to achieve a desired level of throughput, you will want to make calls to the API from parallel tasks. You can control the degree of parallelism by calling `repartition` on the dataframe of pre-batched data. Call `repartition` with the number of parallel tasks you want calling the API. This is actually just one line of code.

```
1 ### Repartition pre-batched df for parallelism of API calls
2 new_df = pre_batched_df.repartition(8)
```